

Wydział Informatyki

Piotr Słupski

Zaawansowana aplikacja webowa do organizacji zadań

Praca: inżynierska
Kierunek: Informatyka
Specjalność: Programowanie
Nr albumu: 7481

Praca wykonana pod kierunkiem:
dr Leszek Grocholski

Wrocław 2026

Spis treści

Wstęp	1
1. Analiza wymagań	4
1.1 Omówienie podobnych aplikacji	5
1.2 Historie użytkownika	6
1.3 Kanban	7
1.4 Najważniejsze funkcje aplikacji	8
2. Frontend	10
2.1 Komponenty w React	12
2.2 React Hooks	14
2.3 TailwindCSS	15
2.4 Typescript	16
3. Backend	18
3.1 NextJS	18
3.2 Routing	19
3.3 Generowanie zawartości strony	19
3.4 API w NextJS	20
3.5 Autoryzacja i autentykacja	21
3.6 Uzasadnienie wyboru NextJS	21
4. Bazy Danych	23
4.1 MySQL	24
4.2 MongoDB	25
4.3 Uzasadnienie wyboru MongoDB	26
5. Architektura aplikacji	27
5.1 Omówienie warstw aplikacji	28
5.2 Diagramy Architektoniczne	29
5.3 Struktura bazy danych aplikacji	32
6. Projekt interfejsu użytkownika	35
6.1 Scenariusze przypadków użycia	35

6.2	Widoki użytkownika aplikacji	40
7.	Implementacja kodu frontendowego	49
7.1	Implementacja widoku kanban	49
7.2	Implementacja sekcji przypiętych elementów dashboardu	50
	Bibliografia	53
	Spis rysunków	54
	Spis tabel	55

Wstęp

Tematem niniejszej pracy inżynierskiej jest opracowanie wraz z implementacją aplikacji internetowej, służącej do organizacji zadań. Zaprojektowane narzędzie skierowane jest do użytkowników indywidualnych, potrzebujących pomocy w sprawnym zarządzaniu zarówno obowiązkami dnia codziennego jak i projektami.

Dynamiczny rozwój technologii internetowych oraz rosnąca liczba obowiązków zawodowych i prywatnych powodują, że efektywne zarządzanie czasem i zadaniami staje się istotnym elementem codziennego funkcjonowania użytkowników. W odpowiedzi na te potrzeby powstają liczne aplikacje, wspierające planowanie pracy, organizację projektów oraz monitorowanie postępów realizacji celów. Jednocześnie rozwój nowoczesnych narzędzi warstwy interfejsu i warstwy serwerowej pozwala na tworzenie wydajnych, skalowalnych i interaktywnych systemów dostępnych z poziomu przeglądarki internetowej.

Inspiracją dla tematu są popularne aplikacje do zarządzania projektami, kładące duży nacisk na organizację pracy zespołowej. Narzędzia takie jak Trello¹, Asana² czy ClickUp³ dostarczają szeroką gamę funkcji, umożliwiających organizację zadań, jednak wiele z nich jest niepotrzebne użytkownikowi indywidualnemu. Z kolei nadmiar zbędnych funkcji może utrudniać użytkownikowi korzystanie z aplikacji. Dlatego też w projekcie zdecydowano się na zaadaptowanie najważniejszych funkcji z wcześniej wymienionych aplikacji, dla potrzeb użytkowników indywidualnych. Tak więc, w prezentowanym projekcie znajdują się uproszczone rozwiązania stosowane w komercyjnych aplikacjach tego typu.

Strona internetowa powinna umożliwiać użytkownikom organizowanie pracy w sposób przejrzysty oraz intuicyjny, z wykorzystaniem funkcji takich jak na przykład przypinanie wybranych treści do widoku strony głównej. Istotnym założeniem projektu jest również zapewnienie bezpieczeństwa danych poprzez wdrożenie mechanizmów logowania, rejestracji oraz autoryzacji użytkowników.

Do celów szczegółowych pracy należy zaprojektowanie graficznego wyglądu interfejsu użytkownika, opracowanie logiki aplikacji zarówno po stronie klienckiej, jak i serwerowej, zaprojektowanie struktury bazy danych oraz implementacja mechanizmów obsługi kont użytkowników. Projekt zakłada wykorzystanie współczesnych technologii webowych⁴, umożli-

¹Por. [1].

²Por. [2].

³Por. [3].

⁴Webowy - z języka angielskiego oznaczający dosłownie "sieciowy". Termin używany powszechnie jako synonim słowa "internetowy".

wiających budowę aplikacji działającej w architekturze klient–serwer⁵, z rozdzieleniem warstwy prezentacji, logiki biznesowej⁶ oraz przechowywania danych.

Zakres realizowanych prac obejmuje analizę dostępnych na rynku rozwiązań, opis odpowiednich narzędzi programistycznych, przygotowanie widoków interfejsu użytkownika oraz przedstawienie implementacji kodu aplikacji. Zakres pracy nie obejmuje publikacji aplikacji w domenie publicznej ani jej komercyjnego wdrożenia. Rezultatem pracy jest funkcjonalna aplikacja internetowa, stanowiąca przykład zastosowania nowoczesnych technologii tworzenia interfejsu i obsługi serwerowej w praktycznej realizacji systemu wspierającego zarządzanie zadaniami. Cele szczegółowe niniejszej pracy obejmują:

- zaprojektowanie graficznego wyglądu interfejsu użytkownika w sposób zapewniający intuicyjną obsługę, czytelność oraz estetykę,
- opracowanie logiki działania systemu po stronie klienckiej, odpowiedzialnej za interakcję z użytkownikiem,
- opracowanie logiki działania systemu po stronie serwerowej, realizującej logikę biznesową oraz komunikację z bazą danych,
- zaprojektowanie struktury bazy danych dostosowanej do przechowywania i przetwarzania informacji o użytkownikach, zadaniach oraz projektach,
- wdrożenie mechanizmów logowania, rejestracji oraz autoryzacji użytkowników, zapewniających bezpieczny dostęp do zasobów aplikacji.

Streszczenie rozdziałów

Rozdział pierwszy przedstawia analizę wymagań projektowanej aplikacji. Omówiono w nim potrzeby użytkownika indywidualnego, porównano podobne narzędzia do organizacji zadań oraz wskazano wymagania funkcjonalne i нефункционалне systemu. W rozdziale opisano również historie użytkownika oraz podstawy wizualnej organizacji zadań w projektach.

Rozdział drugi dotyczy technologii warstwy interfejsu użytkownika zastosowanych podczas realizacji aplikacji. Przedstawiono w nim rolę podstawowych języków używanych do budowy interfejsu, a także opisano wykorzystanie biblioteki komponentowej, mechanizmów pracy ze stanem, narzędzia do stylowania oraz statycznego typowania. Rozdział ten wyjaśnia, w jaki sposób wskazane technologie wspierają budowę interaktywnego i responsywnego interfejsu użytkownika.

Rozdział trzeci opisuje technologie warstwy serwerowej oraz mechanizmy odpowiedzialne za działanie aplikacji po stronie serwera. Szczególną uwagę poświęcono zastosowanemu narzędziu projektowemu, sposobowi obsługi adresów aplikacji, generowaniu

⁵Architektura klient-serwer - model działania systemu, w którym klient wysyła żądania, a serwer je przetwarza i zwraca odpowiedź.

⁶Logika biznesowa - zbiór reguł i mechanizmów aplikacji określających sposób przetwarzania danych oraz realizacji operacji zgodnie z wymaganiami systemu.

zawartości strony, komunikacji między częściami systemu oraz procesom autoryzacji i autentykacji użytkowników.

Rozdział czwarty przedstawia zagadnienia związane z przechowywaniem danych. Omówiono w nim podstawowe cechy baz danych oraz uzasadniono wybór dokumentowego rozwiązania dopasowanego do charakteru danych przetwarzanych przez aplikację.

Rozdział piąty zawiera opis architektury systemu. Przedstawiono w nim podział aplikacji na warstwy, przepływ danych, najważniejsze elementy struktury projektu oraz aktualny opis bazy danych zgodny ze stanem implementacji aplikacji.

Rozdział szósty prezentuje projekt interfejsu użytkownika oraz scenariusze przypadków użycia. Opisano w nim sposób korzystania z głównych funkcji systemu, takich jak obsługa konta użytkownika, praca z notatkami, elementami kontrolnymi, zadaniami i projektami oraz wykorzystanie widoku tablicowego.

Rozdział siódmy opisuje wybrane elementy implementacji kodu warstwy interfejsu użytkownika. Skupiono się w nim na realizacji widoku tablicy projektu oraz sekcji przypiętych elementów dashboardu, ponieważ są to części aplikacji istotne dla codziennej pracy użytkownika oraz wygody korzystania z aplikacji.

1. Analiza wymagań

W dobie rosnącej liczby obowiązków zawodowych, edukacyjnych i prywatnych coraz więcej osób poszukuje skutecznych narzędzi wspierających zarządzanie czasem, planowanie zadań i realizację celów. Wiele aplikacji i systemów do zarządzania zadaniami, takie jak Trello, Asana czy ClickUp zostało zaprojektowanych z myślą o pracy zespołowej oraz dużych organizacjach. W konsekwencji, ich interfejsy bywają skomplikowane, a nadmiar funkcji może przytłaczać i zniechęcać użytkowników indywidualnych, którzy wymagają prostoty, przejrzystości a przede wszystkim szybkiego dostępu do kluczowych funkcji. Użytkownik indywidualny nie potrzebuje licznych widoków listy zadań, zaawansowanej automatyzacji dodawania tasków, czy tworzenia formularzy zespołowych, usprawniających dodawanie wartości do listy. Użytkownik taki potrzebuje natomiast szybko odnaleźć poszukiwane funkcje aplikacji, bądź sprawnie przejść z poziomu strony głównej do projektu, który aktualnie realizuje i sprawdzić jakie rzeczy pozostały mu jeszcze do wykonania.

Na rynku brakuje aplikacji, które w sposób zbalansowany łączą prostotę obsługi z możliwością wyboru sposobu organizacji zadania. Użytkownicy często zmuszeni są korzystać z kilku narzędzi jednocześnie – jednej aplikacji do zarządzania codziennymi zadaniami, innej do planowania bardziej skomplikowanych projektów, a jeszcze innej do tworzenia krótkich notatek. Istnieje więc, potrzeba stworzenia prostej, nowoczesnej aplikacji internetowej, która odpowiadałaby użytkownikom indywidualnym, oferując przejrzysty sposób zarządzania i organizacji różnymi zadaniami.

Narzędzie, takie powinno przede wszystkim:

- być intuicyjne i łatwo dostępne,
- dopasować się do potrzeb indywidualnego użytkownika,
- umożliwiać szybką organizację zadań różnego typu - od codziennych spraw, jak checklisty¹ zakupów, po skomplikowane projekty,
- oferować prosty i motywujący do działania interfejs graficzny.

Użytkownik indywidualny nie potrzebuje licznych nakładek i dodatkowych funkcjonalności aplikacji, usprawniających pracę zespołową. Użytkownik taki ceni sobie najbardziej prostotę. Aplikacja dla niego skierowana powinna przede wszystkim zapewniać

¹Checklista - uporządkowany zestaw elementów do wykonania lub sprawdzenia, w którym poszczególne pozycje można oznaczać jako ukończone.

responsywny, intuicyjny, czytelny i niezawodny interfejs, umożliwiający bezproblemowe nawigowanie po stronie.

1.1 Omówienie podobnych aplikacji

Asana, ClickUp oraz Trello to przykładowe, popularne narzędzia do zarządzania projektami. Z perspektywy pojedynczego użytkownika ich przydatność zależy od złożoności potrzeb oraz preferowanego stylu pracy. **Trello** uchodzi za najprostsze, intuicyjne narzędzie, posiadające najbardziej przejrzysty interfejs. **Asana** natomiast, jest dużo bardziej rozbudowana. Korzystają z niej głównie duże zespoły oraz korporacje. Używa się jej do nadzoru bardziej złożonych projektów. Z kolei **ClickUp**, oferuje za darmo podobny zakres funkcji co płatny plan Asany, będąc przez to wyborem bardziej atrakcyjnym dla wymagających, indywidualnych użytkowników niż **Trello**. Liczne zaawansowane funkcje dostarczane przez ClickUp czynią jednak tą aplikację dość złożoną w obsłudze.

Trello wyróżnia się **prostotą i intuicyjnością** – jest łatwe do opanowania dla początkujących, dzięki czemu idealnie nadaje się do podstawowego porządkowania. Jego jedyna forma prezentacji - kanban² sprzyja szybkiemu organizowaniu prostych projektów. Jednocześnie Trello pozostawia dużo przestrzeni użytkownikom bardziej wymagającym, poprzez możliwość integracji przeróżnych narzędzi zewnętrznych, tak zwanych Power-Upów.

Asana, zaprojektowana została z myślą o pracy zespołowej, dostarczając bardziej ustrukturyzowane podejście do zarządzania zadaniami i projektem. W darmowej wersji obcięte zostały niektóre funkcjonalności, jednak nie posiada ona limitów na ilość zadań, czy projektów. Wiele istotnych funkcji oferowanych przez tę aplikację nie jest przydatne dla użytkownika pojedynczego. Plusem Asany jest stosunkowa prostota w obsłudze oraz czytelność interfejsu, w zestawieniu z ilością oferowanych funkcji.

ClickUp spośród analizowanej trójki, dostarcza najwięcej funkcji za darmo. Zaawansowane widoki, zależności między zadaniami, śledzenie czasu pracy, raporty i wiele wiele więcej sprawia, że jest on aplikacją niemalże kompletną. Jednak taka obfitość narzędzi, odbija się na złożoności interfejsu ClickUp. Jest on mniej intuicyjny, a korzystając z niego trzeba poświęcić wiele więcej uwagi, by odnaleźć poszukiwaną funkcję. Natomiast opanowanie wszystkich możliwości aplikacji może wymagać więcej czasu. Aplikacja dostarcza również rozwiązania systemowe, które maksymalnie ułatwiają skalowalność i dynamiczny wzrost liczby osób pracujących nad projektem. Dlatego też jest ona wysoce zalecana dla startupów lub freelaancerów. Rozpatrując w kontekście indywidualnego użytkownika, to ClickUp jest dużo lepszym wyborem niż Asana, głównie przez ilość usług oferowanych za darmo. Zrozumiałe jest jednak, że wielu użytkownikom może bardziej odpowiadać Trello, ze względu na jego prostotę.

Poniżej znajduje się porównanie najważniejszych wybranych funkcjonalności oferowa-

²Kanban - metoda wizualnego zarządzania pracą, w której zadania przedstawiane są na tablicy podzielonej na etapy realizacji.

nych przez trzy wcześniej wspomniane aplikacje. Warto zaznaczyć, że niektóre płatne funkcjonalności, a w szczególności limity zależą również od planu, który możemy wybrać. W każdej z wymienionych aplikacji istnieje kilka możliwości subskrypcji, kierowanych do różnych grup docelowych.

	ASANA	TRELLO	CLICKUP
GRUPA DOCELOWA	DZIE ZESPOŁY KORPORACJE	ZESPOŁY UŻYTKOWNICY INDYWIDUALNI	ZESPOŁY UŻYTKOWNICY INDYWIDUALNI
INTERFEJS	Skomplikowany, jednak bardzo czytelny natychmiast w ClickUp	Najprostszy i najbardziej intuicyjny. Każdy ma swój własny widok i nie ma potrzeby wywoływania dodatkowych funkcji	Mocno skomplikowany. ClickUp nie ma w tym pierwszoplanowego znaczenia. Pomocna tutaj jest nawigacja z narzędziami i nawigacją
MOŻLIWOŚĆ INTEGRACJI Z INNYMI NARZĘDZIAMI (SLACK, GOOGLE DRIVE)	✓	✓	✓
WIEDZI PROJEKTÓW	• lista • kalendarz • kalendarz (z widokiem) • kalendarz (z widokiem) • kalendarz (z widokiem)	• karta • kalendarz • kalendarz • kalendarz • kalendarz	• lista • kalendarz • kalendarz • kalendarz • kalendarz
AUTOMATYZACJA	• posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań	• posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań	• posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań • posiada narzędzie umożliwiające automatyzację zadań
ZADANIA POWTARZAJĄCE	ClickUp w pełni zaimplementował powtarzające się zadania w określonych terminach	Wymaga jest do tego użycia narzędzia, które umożliwia powtarzanie zadań w określonych terminach	ClickUp w pełni zaimplementował powtarzające się zadania w określonych terminach
FORMULARZE	Posiada możliwość tworzenia formularzy do zbierania informacji niezbędnych do wykonania zadania. Ułatwia to pracę w dużych zespołach	Nie posiada wbudowanego narzędzia, wymagającego integracji z innymi narzędziami	Posiada jak Asana narzędzie do tworzenia formularzy
RAPORTY	Zawiera pełne wsparcie generowania raportów oraz statystyk projektowych. Dodatkowo narzędzie do raportowania, w porównaniu do innych aplikacji jest łatwo dostępne	Generowanie raportów umożliwia integrację z innymi narzędziami (np. Power BI)	Umożliwia tworzenie zautomatyzowanych raportów oraz statystyk projektowych. Dodatkowo narzędzie do raportowania, w porównaniu do innych aplikacji jest łatwo dostępne
DODATKOWE INFORMACJE	Fulla wersja umożliwia budowanie workflow, czyli tworzenie powtarzalnych procesów oraz zautomatyzowanie interakcji między nimi, które pomagają zespołom	-	Aplikacja skupia się na tworzeniu workflow, czyli tworzeniu powtarzalnych procesów oraz zautomatyzowanie interakcji między nimi, które pomagają zespołom

Rys. 1. Tabela, porównująca Trello, Asanę oraz ClickUp.

Ostateczny wybór zależy od indywidualnych preferencji i potrzeb. Dla typowego użytkownika indywidualnego z prostą listą zadań lub małym projektem Trello może okazać się najwygodniejsze, podczas gdy **Asana** sprawdzi się tam, gdzie chcemy bardziej formalnie planować projekty. **ClickUp** będzie najlepszy, w przypadku potrzeby wszechstronnego, elastycznego narzędzia do uporządkowania wielu różnorodnych zadań, w szybko zmieniającym się zespole. Ostatecznie, biorąc pod uwagę użyteczność dla użytkownika indywidualnego, aplikacje ustawiono w kolejności następującej:

1. Trello
2. ClickUp
3. Asana

1.2 Historie użytkownika

W procesie projektowania aplikacji internetowej istotne znaczenie ma dokładne zrozumienie potrzeb przyszłych użytkowników oraz sposobu, w jaki będą oni korzystać z systemu. Jedną z powszechnie stosowanych metod opisu wymagań funkcjonalnych są tzw. historie użytkownika, które przedstawiają oczekiwania wobec aplikacji z perspektywy osoby

korzystającej z oprogramowania. Takie podejście pozwala skupić się nie na szczegółach implementacyjnych, lecz na rzeczywistych celach i problemach, jakie system ma rozwiązywać. Historie użytkownika opisują konkretne scenariusze użycia aplikacji w prosty i zrozumiały sposób. Ułatwia to identyfikację kluczowych funkcjonalności systemu, priorytetyzację wymagań oraz projektowanie interfejsu i logiki działania aplikacji zgodnie z rzeczywistymi potrzebami odbiorców. Dodatkowo stanowią one podstawę do planowania implementacji oraz testowania poprawności działania poszczególnych modułów. Poniżej przedstawiono zestaw pięciu historii użytkownika dla projektowanej aplikacji. Opisują one typowe czynności i odzwierciedlają najważniejsze scenariusze wykorzystania systemu.

1. Jako użytkownik indywidualny, chcę mieć możliwość przypięcia niektórych elementów do strony głównej w celu szybszego dostępu.

2. Jako pracownik biurowy, potrzebuję utworzyć kilka zadań wraz z opisami, określonymi priorytetami oraz czasem ich realizacji, w celu efektywniejszej pracy.

3. Jako głowa rodziny, potrzebuję utworzyć checklistę zakupów, aby w sklepie niczego nie zapomnieć.

4. Jako pracownik nadzorujący dostawy w restauracji, potrzebuję utworzyć notatkę z informacją odnośnie przebiegu dostawy oraz ilości palet zwróconych przez restaurację.

5. Jako student potrzebuję utworzyć projekt, zawierający datę realizacji, opis oraz listę podzadań z możliwością nadawania im różnych statusów i wyświetlania w formie tablicy, by nie pogubić się podczas pisania pracy dyplomowej.

Na podstawie powyższych historii użytkownika opracowano szczegółowe scenariusze przypadków użycia, opisujące sposób interakcji użytkownika z systemem w konkretnych sytuacjach. Znajdują się one w szóstym rozdziale pracy.

1.3 Kanban

Kanban jest metodą organizacji pracy opartą na wizualnym przedstawianiu zadań oraz etapów ich realizacji. W aplikacjach do zarządzania zadaniami przyjmuje najczęściej postać tablicy podzielonej na kolumny, które odpowiadają kolejnym stanom pracy. Zadania są przedstawiane jako karty, a ich przenoszenie pomiędzy kolumnami pozwala szybko ocenić, które elementy są dopiero planowane, które są aktualnie wykonywane, a które zostały już zakończone.

Zaletą tablicy kanban jest jej prostota. Użytkownik nie musi analizować długiej listy zadań ani otwierać każdego elementu osobno, aby sprawdzić postęp pracy. Wystarczy spojrzeć na układ kart w kolumnach. Dzięki temu metoda dobrze sprawdza się zarówno w pracy zespołowej, jak i w pracy indywidualnej, na przykład podczas planowania projektu, nauki, pisania pracy dyplomowej lub organizowania obowiązków domowych.

Przykładowa tablica kanban może wyglądać następująco:

Tabela 1. Przykładowa tablica kanban

Backlog	Do zrobienia	W trakcie	Testowanie	Zrobione
Pomysły na nowe funkcje	Przygotowanie formularza zadania	Implementacja widoku projektu	Sprawdzenie formularza	Konfiguracja logowania
Analiza podobnych aplikacji	Dodanie filtrowania zadań	Połączenie z bazą danych	Test działania checklist	Utworzenie modelu użytkownika

Kolumna *Backlog* przechowuje pomysły lub zadania, które nie zostały jeszcze wybrane do realizacji. Kolumna *Do zrobienia* zawiera elementy zaplanowane jako najbliższe prace. Kolumna *W trakcie* pokazuje zadania aktualnie wykonywane, natomiast *Testowanie* grupuje elementy wymagające sprawdzenia przed uznaniem ich za ukończone. Ostatnia kolumna, *Zrobione*, zawiera zadania zakończone.

W projektowanej aplikacji tablica kanban może zostać wykorzystana przede wszystkim w widoku projektu. Projekt zawiera własne kolumny, a powiązane z nim zadania mogą być przenoszone pomiędzy nimi. Takie rozwiązanie ułatwia kontrolowanie postępu pracy i pozwala użytkownikowi szybko zobaczyć, które zadania wymagają jeszcze uwagi.

1.4 Najważniejsze funkcje aplikacji

Wymagania funkcjonalne i нефункционалне stanowią podstawę do opracowania architektury aplikacji, zaprojektowania interfejsu użytkownika oraz implementacji poszczególnych modułów. Podstawową funkcjonalnością systemu jest umożliwienie użytkownikowi kompleksowego zarządzania elementami organizacyjnymi, którymi są checklisty, notatki, taski oraz projekty. Aplikacja wspiera pełen zestaw operacji CRUD³, co pozwala na swobodne dodawanie elementów, ich edycję oraz usuwanie w zależności od potrzeb użytkownika. Istotnym elementem systemu jest możliwość prezentacji projektów w formie tablicy kanban. Rozwiązanie to pozwala na wizualne przedstawienie stanu realizacji zadań poprzez podział na kolumny, odpowiadające poszczególnym etapom pracy (np. „w trakcie”, „do zmiany”, „zakończone”, „testowane”). Użytkownik może samodzielnie definiować statusy oraz przenosić zadania pomiędzy kolumnami, co wspiera przejrzyste planowanie i monitorowanie postępów. Dodatkowo aplikacja udostępnia panel strony głównej (tzw.dashboard), na którym wyświetlane są przypięte przez użytkownika elementy aplikacji, takie jak na przykład checklisty. Funkcja ta umożliwia szybki dostęp do najważniejszych informacji bez konieczności przeszukiwania całej aplikacji. System pozwala także na wzbogacanie zadań i projektów o dodatkowe dane, takie jak opis, tagi tematyczne, poziom priorytetu czy przewidywany czas realizacji. Dzięki temu użytkownik może lepiej organizować pracę, filtrować elementy oraz

³CRUD (z jęz. ang. Create, Read, Update, Delete) - zestaw podstawowych operacji wykonywanych na danych w systemach informatycznych, obejmujący ich tworzenie, odczyt, modyfikację oraz usuwanie.

ustalać kolejność wykonywania obowiązków. Oprócz funkcjonalności biznesowych istotne znaczenie mają również aspekty jakościowe systemu. Aplikacja umożliwia personalizację poprzez zmianę motywów kolorystycznych oraz przełączanie interfejsu na tryb nocny, co zwiększa komfort użytkowania w różnych warunkach oświetleniowych. Interfejs powinien być minimalistyczny, intuicyjny oraz estetyczny, aby zapewnić łatwość obsługi nawet dla nowych użytkowników. W zakresie bezpieczeństwa przewidziano integrację z zewnętrznymi dostawcami autentykacji, takimi jak Google i GitHub, co pozwala na bezpieczne logowanie bez konieczności przechowywania haseł w aplikacji. System powinien również zapewniać krótkie czasy ładowania, nieprzekraczające 3–4 sekund dla kluczowych operacji, a także efektywną synchronizację danych w czasie rzeczywistym, umożliwiającą natychmiastowe odświeżanie informacji. Architektura aplikacji została zaprojektowana w sposób umożliwiający łatwe skalowanie i rozszerzanie funkcjonalności w przyszłości, co pozwala na dalszy rozwój systemu. Dodatkowo przewidziano stosowanie testów jednostkowych i integracyjnych w celu zapewnienia wysokiej jakości oraz niezawodności kodu.

2. Frontend

Frontend¹ to część strony internetowej lub aplikacji, z którą użytkownik bezpośrednio wchodzi w interakcję. Obejmuje on wszystkie akcje, wykonujące się po stronie przeglądarki. Na stronie internetowej do frontendu zalicza się:

1. Układ strony - poszczególne elementy, jak sekcje, nagłówki, przyciski
2. Wygląd strony - czcionki, kolory, animacje
3. Interakcje - wypełnianie formularzy, klikanie w przycisk, menu nawigacyjne
4. Responsywność - automatyczne dostosowywanie się aplikacji do różnych wielkości ekranu

Fundamentalne narzędzia używane do tworzenia frontendu to HTML², CSS³ oraz JavaScript⁴. HTML to język znaczników, odpowiadający za strukturę strony internetowej. Określa on semantyczny układ treści, takich jak nagłówki, akapity, listy, obrazy czy formularze. HTML stanowi szkielet warstwy frontendowej aplikacji webowej, będąc interpretowanym przez przeglądarkę w celu prezentacji treści użytkownikowi. Odpowiednie ustawienie poprawne i semantyczne rozmieszczenie znaczników HTML ma kluczowe znaczenie dla dostępności cyfrowej, ponieważ umożliwia prawidłową interpretację treści przez czytniki ekranu wykorzystywane przez osoby z niepełnosprawnościami wzroku.

CSS jest językiem służącym do definiowania warstwy prezentacji stron internetowych. Odpowiada za wygląd i układ elementów HTML, w tym kolory, czcionki, odstępy, pozycjonowanie oraz animacje. Dzięki określonymu mechanizmowi dziedziczenia, CSS umożliwia spójne oraz elastyczne zarządzanie stylem w obrębie całej aplikacji webowej, a jego poprawne zastosowanie wspiera czytelność interfejsu oraz dostępność treści dla użytkowników.

JavaScript natomiast jest językiem programowania, umożliwiającym nadanie stronie interaktywności i dynamicznego charakteru. Pozwala na reagowanie na działania użytkownika,

¹Frontend - termin pochodzący z języka angielskiego, będący złożeniem słów “front” (przód, część widoczna) oraz “end” (część, warstwa), odnoszący się do tej części aplikacji, która jest bezpośrednio dostępna dla użytkownika.

²HTML - z jęz. ang. Hypertext Markup Language, czyli “hipertekstowy język znaczników”. Por. [4].

³CSS - z jęz. ang. Cascading Style Sheets, oznaczające “kaskadujące arkusze stylów”. Por. [4].

⁴JavaScript - język programowania wykorzystywany do tworzenia interaktywnych elementów stron internetowych. Por. [4].

modyfikowanie treści oraz struktury dokumentu HTML w czasie rzeczywistym, a także komunikację z serwerem bez konieczności przeładowywania strony. JavaScript stanowi kluczowy element nowoczesnych aplikacji webowych, integrując warstwę prezentacji z logiką działania interfejsu użytkownika.

Wraz ze wzrostem złożoności współczesnych interfejsów użytkownika, wynikła potrzeba rozbudowy i rozszerzania podstawowych technologii frontendowych.

Wczesne zastosowania HTML, CSS i JavaScript ograniczały się głównie do prostych, statycznych stron. Jednak wraz ze wzrostem ilości aplikacji interaktywnych, pojawiła się konieczność lepszej organizacji kodu, ponownego wykorzystania komponentów oraz efektywniejszego zarządzania stanem aplikacji. Dynamiczny rozwój stron internetowych względem ich funkcjonalności i użyteczności zmusił programistów do usprawnienia wcześniej wymienionych języków, w celu sprostania rosnącym oczekiwaniom użytkowników. Biblioteki powstały jako odpowiedź na te wyzwania, umożliwiając realizację dużo bardziej ambitnych i zaawansowanych aplikacji internetowych.

Biblioteki programistyczne to zbiory gotowych funkcji, klas oraz komponentów, które rozszerzają możliwości języków programowania i ułatwiają realizację określonych zadań. Ich celem jest ponowne wykorzystanie sprawdzonych rozwiązań, by nie wymyślać koła na nowo. Pozwalają one na skrócenie czasu tworzenia oprogramowania oraz zwiększenie czytelności i niezawodności kodu. W kontekście technologii frontendowych biblioteki wspierają m.in. budowę interfejsu użytkownika, obsługę zdarzeń oraz zarządzanie danymi po stronie klienta. Najważniejszym przykładem jest tutaj React⁵, czyli biblioteka języka JavaScript.

Jak można wyczytać na oficjalnej stronie dokumentacji biblioteki, dostarcza ona narzędzia, pozwalająca między innymi na tworzenie komponentów nadających się do ponownego użycia w różnych sekcjach aplikacji. W ten sposób można stworzyć na przykład przycisk, bądź nagłówek, bez potrzeby kilkukrotnego przepisywania kodu, gdy chcemy by nasz dokument HTML zawierał kilka takich samych elementów. Takie podejście nie tylko oszczędza czas, ale również znacznie poprawia przejrzystość kodu. Właśnie z tego powodu w pracy zdecydowano się na skorzystanie z tej biblioteki. Obok bibliotek, rozwinęło się dużo narzędzi programistycznych zwanych frameworkami⁶. Różnica pomiędzy biblioteką a frameworkiem dotyczy przede wszystkim stopnia kontroli nad przepływem aplikacji. Biblioteka dostarcza zestaw narzędzi i funkcji, które są wywoływane bezpośrednio przez programistę w wybranym miejscu kodu. Framework natomiast narzuca określoną strukturę projektu oraz sposób działania aplikacji, przejmując kontrolę nad jej przepływem i wymuszając pewne ramy, których programista musi przestrzegać.

⁵Por. [5]: *React the library for web and native interfaces*, <https://react.dev> (dostęp: 28.01.2026).

⁶Framework - z języka angielskiego, jest złożeniem słów "frame" (rama) oraz "work" (praca, struktura). Dosłownie oznacza "ramę konstrukcyjną" lub "szkielet".

2.1 Komponenty w React

Komponenty w React stanowią podstawowy element budowy interfejsu użytkownika i umożliwiają podział aplikacji na mniejsze, niezależne moduły. Takie podejście redukuje powielanie kodu oraz ułatwia jego testowanie i utrzymanie. Modułowa struktura aplikacji reactowej pozwala na lepszą organizację, zwiększając zarówno czytelność kodu, jak i skalowalność projektu. Jednak wyżej wspomniane funkcje Reacta nie byłyby możliwe dzięki nakładce do języka JavaScript zwanej JavaScript Extension - w skrócie JSX.

Jako, że HTML jest językiem deklaratywnym służącym do opisu struktury dokumentu, natomiast JavaScript odpowiada za logikę i manipulację tą strukturą. W klasycznym podejściu HTML i JavaScript są rozdzielone - pierwszy język definiuje widok, a drugi modyfikuje go poprzez interakcję z DOM⁷. React poprzez JSX umożliwia deklaratywne tworzenie komponentów w skryptach. By zmanipulować DOM'em przy użyciu czystego JavaScriptu należy pierw użyć szeregu przygotowanych przez twórców JSa⁸ funkcji. JSX pozwala nam ominąć ten niewygodny oraz czasochłonny proces i pisać wewnątrz plików JavaScripta kod HTML deklaratywnie. To znacznie ułatwia definiowanie, utrzymanie oraz ponowne wykorzystanie komponentów. Dodatkowo składnia JSX jest podczas procesu budowania aplikacji transpilowana⁹ do czystego JavaScriptu, dzięki czemu React może efektywnie zarządzać komponentami, ich stanem oraz ponownym wykorzystaniem. Poniższy przykład wyjaśnia, dlaczego JSX znacząco upraszcza tworzenie komponentów w React.

```
function Button() {  
  const btn = document.createElement("button");  
  btn.textContent = "Kliknij";  
  btn.onclick = () => alert("Kliknięto");  
  return btn;  
}
```

Rys. 2. fragment kodu przedstawiający definiowanie komponentu *Button* w czystym JavaScript, bez nakładki *JSX*

Jak widać na wyżej zamieszczonej grafice, w czystym JavaScriptcie komponent można zbudować wyłącznie poprzez imperatywne tworzenie elementów. Takie podejście jest mało czytelne, trudne w utrzymaniu i nie skaluje się dobrze w przypadku złożonych interfejsów.

⁷DOM - z jęz. ang. Document Object Model, to hierarchiczna reprezentacja struktury dokumentu HTML w postaci drzewa obiektów. Umożliwia on językom programowania, takim jak JavaScript, dynamiczny dostęp do elementów strony oraz ich modyfikację w czasie działania aplikacji.

⁸JS - skrót od JavaScript

⁹Transpilacja - to proces przekształcania kodu z jednej wersji jakiegoś języka na inną

```
function Button() {
  return <button onClick={() => alert("Kliknięto")}>Kliknij</button>;
}
```

Rys. 3. fragment kodu przedstawiający deklaratywne definiowanie komponentu Button dzięki JSX

Wyżej przedstawiono, jak prosto w sposób deklaracyjny dodaje się elementy HTMLa, dzięki bibliotece React. Na powyżej zamieszczonych przykładach można zaobserwować, jak ogromna jest różnica pomiędzy wspomnianymi metodami obsługi znaczników HTML w JavaScriptcie. W React wyróżnia się dwa podstawowe typy komponentów - klasowe oraz funkcyjne, różniące się zarówno składnią, jak i sposobem zarządzania logiką aplikacji. Komponenty klasowe opierają się na klasach wprowadzonych do JavaScript wraz z aktualizacją ES6¹⁰. Dziedziczą one po klasie **React.Component**, a zarządzanie stanem oraz cyklem życia¹¹ komponentu odbywa się za pomocą metod takich jak **constructor**, **componentDidMount**, czy **componentDidUpdate**. Taki model prowadzi często do bardziej rozbudowanego i mniej czytelnego kodu, zwłaszcza w przypadku złożonych komponentów. Poniżej widnieje przykład komponentu klasowego o nazwie "Button". Komponent ten renderuje¹² przycisk, którego treść oraz obsługa zdarzenia kliknięcia są przekazywane z zewnątrz za pomocą parametru "props", co jest standardem zarówno jeśli chodzi o komponenty klasowe jak i funkcyjne.

```
import React from "react";

class Button extends React.Component {
  render() {
    return (
      <button onClick={this.props.onClick}>
        {this.props.title}
      </button>
    );
  }
}

export default Button;
```

Rys. 4. fragment kodu przedstawiający klasowy komponent "Button"

Komponenty funkcyjne są zwykłymi funkcjami JavaScript, przyjmującymi dane wejściowe za pomocą "props" i zwracają strukturę interfejsu użytkownika. Początkowo były

¹⁰ES6 - skrót od "ECMAScript 2015", jest to szósta edycja standardu języka JavaScript, która znacząco rozszerzyła możliwości języka.

¹¹cykl życia komponentu - zbiór kolejnych etapów istnienia komponentu w aplikacji, obejmujący jego generowanie, aktualizację w odpowiedzi na zmiany danych oraz usuwanie z drzewa komponentów.

¹²renderowanie - generowanie, ładowanie

one ograniczone do komponentów prezentacyjnych, jednak wprowadzenie mechanizmu React Hooks¹³, rozszerzyło ich zastosowanie do pełnoprawnych elementów logiki aplikacji. Dzięki prostszej składni i lepszej modularności komponenty funkcyjne są obecnie preferowanym i rekomendowanym podejściem w nowoczesnych aplikacjach. Poniżej zamieszczono przykład tego samego komponentu “Button”, lecz w formie funkcji.

```
import React from "react";

function Button({ onClick, title }) {
  return (
    <button onClick={onClick}>
      {title}
    </button>
  );
}

export default Button;
```

Rys. 5. fragment kodu przedstawiający funkcyjny komponent “Button”

2.2 React Hooks

Mechanizm hooków umożliwia przechowywanie danych pomiędzy kolejnymi wywołaniami funkcji komponentu, mimo że sama funkcja jest wykonywana od nowa przy każdym renderowaniu. Każdy hook jest wywoływany bezpośrednio w ciele funkcji komponentu, dzięki czemu React może jednoznacznie przypisać jego działanie do danego komponentu oraz zachować odpowiednie wartości pomiędzy kolejnymi wywołaniami tej funkcji. Kolejność wywołań hooków ma kluczowe znaczenie, ponieważ na jej podstawie React identyfikuje, które dane należą do którego hooka.

W odróżnieniu od zwykłych zmiennych lokalnych, które są tracone po zakończeniu wykonania funkcji, wartości zarządzane przez hooki są przechowywane poza samą funkcją, lecz udostępniane jej ponownie przy każdej aktualizacji interfejsu użytkownika. Dzięki temu komponent funkcyjny może być ponownie wykonywany, zachowując ciągłość danych oraz logiki aplikacji. W JavaScriptcie zasięg funkcji określa obszar kodu, w którym dana zmienna lub funkcja jest dostępna. W przypadku komponentów funkcyjnych oznacza to, że wszystkie zmienne, Hooki oraz funkcje pomocnicze zadeklarowane wewnątrz komponentu są dostępne wyłącznie w obrębie jego ciała. Zapewnia to, że stan oraz logika komponentu pozostają hermetyczne i nie kolidują z innymi komponentami, co sprzyja modularności, przewidywalności oraz bezpieczeństwu struktury aplikacji. Zarządzanie stanem aplikacji w komponencie polega na przechowywaniu oraz aktualizowaniu danych, które mają bezpośredni wpływ na wy-

¹³Hooks - z jęz. ang. “zaczepy”, w React są to funkcje, pozwalające na dostęp do tych samych danych w obrębie całego zakresu funkcji.

świetlany interfejs użytkownika. Stan reprezentuje bieżące parametry komponentu, takie jak wartości formularza, licznik, widoczność elementów, czy dane pobrane z serwera. Zmiana stanu powoduje ponowną aktualizację widoku komponentu w celu odzwierciedlenia nowych danych. W komponentach funkcyjnych stan jest obsługiwany za pomocą funkcji stanu, składającej się z aktualnej wartości stanu oraz funkcji służącej do jej zmiany. Funkcja ta nie modyfikuje stanu bezpośrednio, lecz informuje Reacta o konieczności jego aktualizacji, co pozwala zachować spójność danych i przewidywalność działania aplikacji.

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Licznik: {count}
    </button>
  );
}

export default Counter;
```

Rys. 6. przykład funkcji zarządzającej stanem komponentu "Counter"

W powyższym przykładzie:

- zmienna **count** przechowuje aktualny stan licznika,
- funkcja **setCount** odpowiada za aktualizację stanu,
- wywołanie funkcji **setCount** powoduje zmianę danych oraz odświeżenie widoku komponentu na stronie

Takie podejście zapewnia enkapsulację danych wewnątrz komponentu oraz precyzyjne sterowanie jego zachowaniem. Dzięki hookom zarządzanie stanem staje się czytelne i ściśle powiązane z zakresem funkcji komponentu, co ułatwia rozwój i utrzymanie aplikacji frontendowych.

2.3 TailwindCSS

TailwindCSS¹⁴ jest frameworkiem służącym do stylowania interfejsów użytkownika za pomocą gotowych klas CSS, używanych bezpośrednio w znacznikach HTML lub komponentach Reactowych. Tailwind umożliwia szybkie budowanie spójnych i responsywnych

¹⁴Por. [6]: *Strona Tailwindcss*, <https://tailwindcss.com> (dostęp: 28.01.2026).

layoutów bez konieczności pisania własnych arkuszy stylów dla każdego komponentu. Technologia ta jest stosowana w celu zwiększenia produktywności, ograniczenia duplikacji kodu CSS oraz łatwiejszego skalowania warstwy prezentacyjnej aplikacji frontendowej.

```
1 <div class="flex flex-col items-center gap-6 p-7 md:flex-row md:gap-8 rounded-2xl">
2   <div>
3     
4   </div>
5   <div class="flex items-center md:items-start">
6     <span class="text-2xl font-medium">Class Warfare</span>
7     <span class="font-medium text-sky-500">The Anti-Patterns</span>
8     <span class="flex gap-2 font-medium text-gray-600 dark:text-gray-400">
9       <span>No. 4</span>
10      <span>•</span>
11      <span>2025</span>
12    </span>
13  </div>
14 </div>
```

Rys. 7. przykładowe użycie TailwindCSS, pochodzące z oficjalnej strony frameworka
(dostęp: 28.01.2026)

Jak widać na obrazie, całe stylowanie realizowane jest poprzez tak zwane klasy narzędziowe (po angielsku utility classes) Tailwinda, bez użycia osobnych plików CSS. Dla przykładu - klasa “flex” nadana znacznikowi “div” w piątej linii kodu, jest tożsama z CSSowym “display: flex;”. W niniejszej pracy inżynierskiej Tailwinda użyto przy projektowaniu aplikacji, z następujących powodów:

- dzięki niemu nie trzeba definiować własnych klas CSS,
- umożliwia szybkie tworzenie responsywnych i spójnych komponentów,
- eliminuje potrzebę przechodzenia do osobnych plików stylów w celu sprawdzenia struktury wizualnej znacznika lub komponentu.

2.4 Typescript

Kolejną technologią zastosowaną przy projekcie aplikacji jest TypeScript, czyli statycznie typowany nadzbiór języka JavaScript. Rozszerza on JSa o system typów, interfejsy oraz mechanizmy kontroli poprawności kodu na etapie kompilacji. Kod TypeScript nie jest bezpośrednio interpretowany przez przeglądarkę, lecz przed uruchomieniem musi zostać transpilowany do standardowego JavaScriptu, zachowując pełną kompatybilność ze środowiskiem wykonawczym. Głównym celem stosowania TypeScript jest zwiększenie bezpieczeństwa, czytelności oraz przewidywalności kodu. Statyczne typowanie pozwala wykrywać błędy już na etapie pisania kodu a nie podczas działania aplikacji. TypeScript znacząco ułatwia pracę w

dużych projektach oraz zespołach programistycznych, ponieważ jasno definiuje struktury danych oraz oczekiwane typy argumentów. Ponadto redukuje ryzyko błędów logicznych trudnych do wykrycia w czystym JavaScriptcie i dobrze integruje się z nowoczesnymi frameworkami frontendowymi. Brak TypeScriptu w większych aplikacjach pisanych w JavaScript może prowadzić do szeregu problemów. Przykładowo, JS pozwala na swobodne przekazywanie danych dowolnego typu, co może skutkować błędami w czasie działania aplikacji, np. wywołaniem metod na nieprawidłowym typie danych. Również wiele problemów, takich jak błędnie zapisane wyrazy w nazwach pól obiektów, czy nieprawidłowe argumenty funkcji, ujawnia się dopiero podczas działania aplikacji. W czystym JavaScriptcie brakuje też mechanizmu, wymuszającego spójność zmian w całym projekcie, przez co zmiana struktury danych wiąże się z większym ryzykiem niezamierzonych błędów. Jednak najważniejszą cechą projektów, wykorzystujących Typescript jest zdecydowanie dużo wyższa czytelność i przewidywalność kodu. Jawnie zdefiniowane typy ułatwiają zrozumienie, jakie dane są przetwarzane przez funkcje oraz komponenty, co znacznie wpływa na utrzymanie aplikacji oraz wdrażanie nowych programistów do projektu. Jak dokładnie działa TypeScript można wytłumaczyć na następującym przykładzie. Tworzony jest komponent odpowiedzialny za wyświetlanie na stronie danych użytkownika - takich jak imię, adres e-mail oraz wiek. W TypeScriptie definowany jest typ lub interfejs, który jednoznacznie określa, jaką strukturę muszą mieć parametry przyjmowane przez komponent - n.p. imię i adres e-mail muszą być zmienną typu "string", a wiek - "number". Następnie komponent lub funkcja przyjmująca dane użytkownika deklaruje, że oczekuje dokładnie takiego typu danych. Dzięki temu TypeScript już na etapie pisania kodu sprawdza, czy przekazywane wartości są poprawne. Jeżeli programista spróbuje przekazać zmienną typu "number" zamiast "string" lub pominie wymagane pole, błąd zostanie wykryty jeszcze przed uruchomieniem aplikacji. W trakcie dalszego rozwoju projektu, gdy struktura danych użytkownika ulegnie zmianie, TypeScript automatycznie wskaże wszystkie miejsca w aplikacji, które wymagają aktualizacji. Zintegrowane środowiska programistyczne¹⁵, posiadają możliwość integracji z TypeScriptem poprzez wyświetlanie na bieżąco powiadomień i ostrzeżeń o potencjalnych błędach wykrywanych przez ten język.

¹⁵Zintegrowane środowisko programistyczne - w skrócie IDE, to zestaw narzędzi oraz konfiguracji wykorzystywanych do tworzenia oprogramowania, obejmujący m.in. edytor lub środowisko IDE

3. Backend

Backend¹ to warstwa aplikacji działająca po stronie serwera i odpowiedzialna za przetwarzanie danych oraz komunikację z bazą danych lub innymi systemami. Backend nie jest bezpośrednio widoczny dla użytkownika końcowego. Obsługuje m.in. autoryzację, walidację² danych, realizację reguł biznesowych oraz udostępnianie danych frontendowi za pomocą API³.

API natomiast to zestaw reguł, metod i struktur danych umożliwiających komunikację pomiędzy różnymi systemami informatycznymi lub ich częściami. Określa, w jaki sposób aplikacje mogą wymieniać dane oraz wywoływać określone funkcje, bez konieczności znajomości wewnętrznej implementacji drugiej strony. API pełni rolę pośrednika pomiędzy frontendem a backendem. Frontend wysyła żądania lub zapytania, natomiast backend przetwarza je, zwracając później odpowiedź.

3.1 NextJS

Next.js⁴ jest frameworkiem opartym na bibliotece React, przeznaczonym do tworzenia pełnostosowych⁵ aplikacji webowych. Interfejs użytkownika budowany jest przy użyciu komponentów Reacta, natomiast Next.js dostarcza dodatkowe funkcje oraz mechanizmy optymalizacyjne. Framework automatycznie konfiguruje narzędzia niskiego poziomu, takie jak bundlery⁶ oraz kompilatory⁷, dzięki czemu programista może skupić się na tworzeniu aplikacji i szybkim wdrażaniu jej do użytku.

Next rozszerza możliwości Reacta o funkcje takie jak renderowanie zawartości po stronie serwera, czy routing⁸ oparty na strukturze plików. Umożliwia również budowę aplikacji

¹Backend -termin pochodzący z języka angielskiego, będący złożeniem słów "back" (tył, zaplecze) oraz end (część, warstwa), odnoszący się do części aplikacji niedostępnej dla użytkownika.

²Walidacja - proces sprawdzania poprawności, kompletności oraz zgodności danych wejściowych z określonymi regułami i wymaganiami systemu.

³API - skrót z języka angielskiego, oznaczający Application Programming Interface.

⁴Por. [7]: *NextJS Docs*, <https://nextjs.org/docs> (dostęp 29.01.2026).

⁵Pełnostosowy - Stos to określenie powszechnie stosowane w języku programistycznym jako właśnie narzędzi użytych do produkcji aplikacji. Full-stack to termin z języka angielskiego, oznaczający technologie zarówno frontendowe, jak i backendowe.

⁶Bundler - narzędzie programistyczne służące do łączenia wielu plików źródłowych w jeden lub kilka zoptymalizowanych pakietów, które mogą być uruchamiane w przeglądarce.

⁷Kompilator - program tłumaczący kod źródłowy napisany w jednym języku programowania na inną postać, umożliwiając jego uruchomienie w danym środowisku.

⁸Routing - mechanizm zarządzania nawigacją w aplikacji, umożliwiający użytkownikowi poruszanie się pomiędzy różnymi częściami systemu.

przyjaznych dla wyszukiwarek internetowych, integrując w jednym narzędziu frontend oraz wybrane elementy backendu. Dzięki temu upraszcza architekturę projektu i jest często wykorzystywany do tworzenia aplikacji produkcyjnych oraz stron o wysokich wymaganiach wydajnościowych. Dodatkowo Next dostarcza gotowy komponent “Image” - usprawniający ładowanie obrazów na stronie oraz “Link” - zapewniający nawigację pomiędzy stronami aplikacji w sposób zoptymalizowany i wydajny.

3.2 Routing

Routing w Next.js stanowi mechanizm odpowiedzialny za mapowanie adresów URL na konkretne widoki aplikacji. W przeciwieństwie do klasycznych aplikacji React, w których konfiguracja tras wymaga dodatkowych bibliotek i ręcznego definiowania reguł nawigacji, Next.js wykorzystuje routing oparty na strukturze plików - z jęz. ang. file-based routing. Ścieżki URL są automatycznie generowane na podstawie organizacji katalogów i plików w projekcie. Podstawową zasadą działania tego mechanizmu, jest przypisanie każdemu plikowi w katalogu pages lub app, odpowiadającej mu trasy. Przykładowo - plik “about.js” generuje ścieżkę “/about”, natomiast plik umieszczony w podkatalogu “blog/post.js”, odpowiada adresowi “/blog/post”. Takie rozwiązanie upraszcza konfigurację projektu, zwiększa czytelność struktury aplikacji oraz ogranicza konieczność tworzenia dodatkowych plików konfiguracyjnych. Next.js obsługuje również routing dynamiczny, który umożliwia tworzenie stron o zmiennych parametrach, np. dla wpisów blogowych czy profili użytkowników. Realizowane jest to poprzez specjalną składnię nazw plików, co pozwala na generowanie wielu widoków na podstawie jednego szablonu komponentu. Aby utworzyć dynamiczną ścieżkę, zmieniającą się w zależności od parametru “id” użytkownika, należy w katalogu “users” utworzyć plik o nazwie [id].js. Wewnątrz komponentu parametr “id” można odczytać ze specjalnego obiektu routing. Przykładowo, pojedynczy komponent może generować widok profilu na podstawie identyfikatora przekazanego w adresie URL. Takie podejście pozwala na tworzenie wielu stron przy użyciu jednego szablonu, co zwiększa modularność oraz skalowalność aplikacji. Dodatkowo framework wspiera nawigację po stronie klienta, realizowaną za pomocą komponentu “Link”, dzięki czemu przejścia pomiędzy stronami odbywają się bez pełnego przeładowania dokumentu. Rozwiązanie to zwiększa wydajność oraz poprawia płynność działania interfejsu użytkownika. Zastosowanie wbudowanego systemu routing w Next.js upraszcza proces tworzenia aplikacji, redukuje ilość kodu konfiguracyjnego oraz sprzyja zachowaniu spójnej i logicznej struktury projektu, co ma istotne znaczenie w większych i długoterminowych przedsięwzięciach programistycznych.

3.3 Generowanie zawartości strony

Domyślnie NextJS używa serwerowych komponentów Reacta. Wykonują się one najpierw po stronie serwera, a następnie ich wynik wysyłany jest do klienta.

Komponenty te zapewniają wsparcie dla JSowych obiektów typu Promise, umożliwiając wykonywanie zadań asynchronicznych, bez konieczności używania dodatkowych bibliotek czy hooków Reactowych. Natomiast pobieranie danych zawsze odbywa się asynchronicznie. Ponieważ komponenty serwerowe działają po stronie serwera, możliwe jest bezpośrednie wykonywanie zapytań do bazy danych bez konieczności korzystania z dodatkowej warstwy API. Pozwala to ograniczyć ilość kodu, który trzeba pisać i utrzymywać. Pobrane dane można generować i przetwarzać na dwa różne sposoby. Renderowanie statyczne polega na generowaniu treści na etapie budowania aplikacji lub podczas jej ponownej walidacji, a następnie dostarczaniu gotowych stron użytkownikom, co zwiększa wydajność i zmniejsza obciążenie serwera oraz sprzyja optymalizacji SEO⁹. Renderowanie dynamiczne natomiast, odbywa się przy każdym żądaniu użytkownika, umożliwiając prezentację danych aktualnych w czasie rzeczywistym oraz treści spersonalizowanych. Pierwsze podejście sprawdza się w przypadku stron o stałej zawartości, natomiast drugie jest preferowane w aplikacjach, wymagających częstych aktualizacji danych, takich jak panele administracyjne czy profile użytkowników. Buforowanie danych polega na tymczasowym przechowywaniu wyników wcześniej wykonanych operacji, takich jak zapytania do bazy danych lub odpowiedzi z API, w celu ich ponownego wykorzystania bez konieczności ponownego przetwarzania. Dzięki temu kolejne żądania mogą zostać obsłużone szybciej, ponieważ dane są odczytywane z pamięci podręcznej zamiast generowane od podstaw. Mechanizm ten zmniejsza obciążenie serwera, skraca czas odpowiedzi aplikacji oraz poprawia ogólną wydajność systemu. Równie istotnym elementem optymalizacji w NextJS, jest deduplikacja zapytań, czyli eliminacja wielokrotnego pobierania tych samych danych. Jeżeli kilka komponentów wymaga tych samych informacji, system wykonuje zapytanie tylko raz, a następnie współdzieli wynik pomiędzy wszystkimi. Zmniejsza to obciążenie serwera, skraca czas odpowiedzi oraz poprawia efektywność wykorzystania zasobów, co ma szczególne znaczenie w rozbudowanych aplikacjach.

3.4 API w NextJS

Ścieżki API w NextJS to mechanizm, umożliwiający definiowanie punktów końcowych¹⁰ API bezpośrednio w projekcie Next.js. Endpointy (z jęz. ang. punkty końcowe) można tworzyć poprzez umieszczenie plików w katalogu “pages/api”, a każda taka funkcja obsługuje żądania HTTP i zwraca odpowiedź w modelu “request/response”¹¹. W nowszym podejściu, odpowiednikami ścieżek API są tzw. Route Handlers (z jęz. ang. obsługa ścieżek), definiowane w katalogu “app” i pliku “route.js”. W tym modelu obsługa metod HTTP reali-

⁹SEO (z jęz. ang. Search Engine Optimization) – proces optymalizacji stron internetowych, mający na celu poprawę ich widoczności w wynikach wyszukiwarek internetowych poprzez dostosowanie treści, struktury oraz wydajności serwisu do wymagań algorytmów indeksujących.

¹⁰Punkt końcowy - reprezentowany jest przez określony adres URL oraz metodę HTTP, udostępniając konkretną funkcjonalność lub zasób systemu. Umożliwia komunikację pomiędzy klientem a serwerem.

¹¹Interfejs typu request/response – model komunikacji pomiędzy klientem a serwerem, w którym klient wysyła żądanie (request) zawierające określone dane lub operację, a serwer przetwarza je i zwraca odpowiedź (response) z wynikiem działania, danymi lub informacją o błędzie.

zowana jest przez eksport funkcji takich jak “GET”, “POST”, “PUT” czy “DELETE”, co pozwala tworzyć endpointy w sposób bardziej zbliżony do standardowych interfejsów typu “request/response”. Zastosowanie API Routes obejmuje m.in. obsługę formularzy, operacje CRUD, integracje z usługami zewnętrznymi, czy realizację logiki po stronie serwera, która nie powinna być wykonywana w przeglądarce. Istotną zaletą jest fakt, że kod endpointów nie trafia do klienta, dzięki czemu możliwe jest bezpieczniejsze przetwarzanie danych oraz użycie zmiennych środowiskowych po stronie serwera.

3.5 Autoryzacja i autentykacja

W Next.js autentykację i autoryzację można w prosty sposób wdrożyć poprzez integrację z dostawcą OAuth¹² przy użyciu biblioteki Auth.js. Rozwiązanie to pozwala skonfigurować logowanie przez zewnętrznych dostawców, takich jak Google, czy Github. Biblioteka automatycznie obsługuje sesje użytkownika i przekazywanie tokenów. Po zalogowaniu aplikacja może kontrolować dostęp do zasobów na podstawie informacji z sesji, np. ról lub uprawnień, ograniczając dostęp do wybranych stron i endpointów API.

3.6 Uzasadnienie wyboru NextJS

W niniejszej pracy dyplomowej, zdecydowano się na skorzystanie z NextJSa z uwagi na szereg narzędzi, udostępnianych przez niego, które wymieniono we wcześniejszych podrozdziałach. Next.js stanowi doskonałe środowisko do budowy aplikacji internetowych opartych o React, zapewniając jednocześnie wygodne narzędzia typowe dla warstwy backendowej. Poniżej przedstawiono kluczowe argumenty przemawiające za jego wykorzystaniem. Jedną z istotnych zalet Next.js jest wsparcie dla różnych strategii renderowania, co pozwala dopasować sposób generowania treści do charakteru poszczególnych ekranów aplikacji. W kontekście aplikacji do organizacji zadań większość treści znajduje się za panelem logowania, jednak nadal istnieją elementy, które mogą korzystać z poprawionej indeksowalności oraz szybszego wyświetlania strony, jak chociażby strona logowania. Renderowanie po stronie serwera umożliwia dostarczenie kompletnego HTML już na etapie odpowiedzi serwera, co ułatwia pracę algorytmom indeksującym, a także skraca czas potrzebny do pierwszego wyświetlenia treści. Dodatkowo Next.js ułatwia zarządzanie metadanymi strony(np. tytuł, opis), co przekłada się na spójne i przewidywalne zachowanie aplikacji w przeglądarkach oraz w serwisach społecznościowych. Z punktu widzenia pracy inżynierskiej ważne jest to, że framework oferuje mechanizmy SEO wbudowane w standardowy sposób tworzenia aplikacji, bez konieczności ręcznego konfigurowania. Aplikacja do zarządzania zadaniami posiada ogromny potencjał do konsekwentnego rozwijania jej wraz z dodawaniem nowych funkcjonalności. Next.js

¹²OAuth - z jęz. ang. Open Authorization, to standard protokołu autoryzacji umożliwiający aplikacjom uzyskanie dostępu do zasobów użytkownika w innym systemie bez konieczności przekazywania jego hasła. Zamiast tego wykorzystywane są specjalne tokeny dostępu, które określają zakres przyznanych uprawnień.

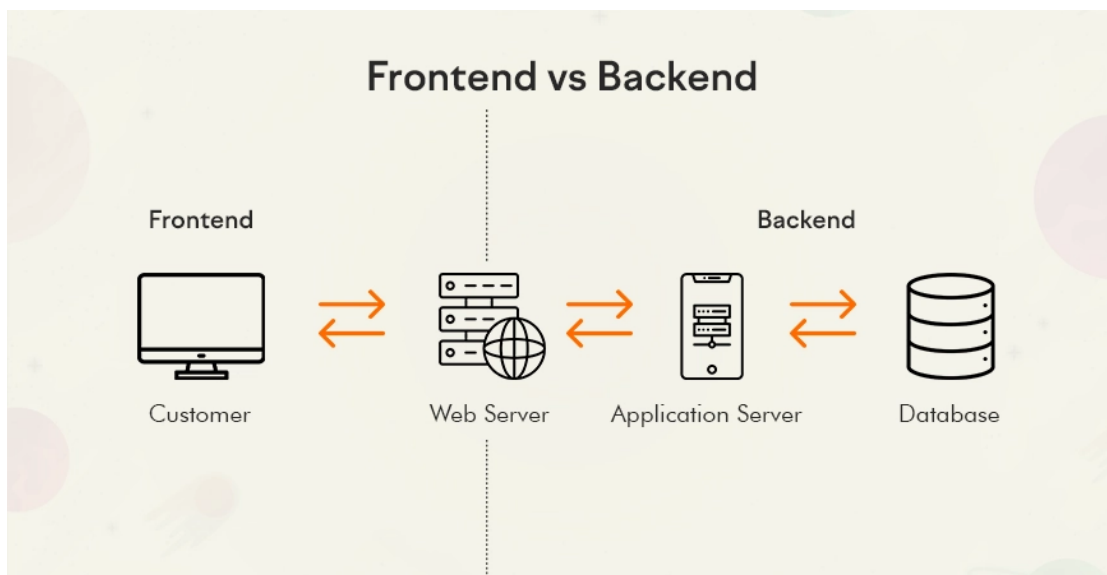
sprzyja umożliwić skalowalność na kilku poziomach, lecz przede wszystkim zapewnia czytelną strukturę projektu, dzięki routingowi opartemu o system plików wraz z możliwościami wydzielania modułów. Next.js pozwala rozwijać aplikację etapami, utrzymując spójny model wytwarzania, natomiast wraz ze wzrostem złożoności projektu nie zachodzi potrzeba migracji na inne narzędzia. Kolejnym argumentem jest możliwość budowy zarówno warstwy prezentacji, jak i elementów backendowych w ramach jednego frameworka. Next.js dostarcza mechanizmy takie jak endpointy API, middleware¹³ oraz integrację z logiką serwerową, co jest szczególnie korzystne w wymagających aplikacjach. Takie podejście mocno skraca czas implementacji, ponieważ eliminuje konieczność utrzymywania osobnego projektu backendowego. Zmniejsza to również ryzyko niespójności kontraktów API oraz upraszcza jej wdrażanie, gdyż aplikacja może być publikowana jako całość. Istotną zaletą Nexta jest również integracja z biblioteką Auth.js¹⁴. Pozwala to implementować zabezpieczenia zarówno po stronie klienta - jak warunkowe renderowanie widoków, oraz po stronie serwera - ochrona endpointów API, czy walidacja sesji w middleware. Dzięki temu kontrola dostępu nie jest oparta wyłącznie o logikę frontendową, lecz może być egzekwowana na poziomie serwera, co zwiększa bezpieczeństwo rozwiązania. Zastosowanie Auth.js eliminuje również konieczność budowania całego mechanizmu logowania od podstaw. Stworzenie logiki, zajmującej się obsługiwaniem sesji, tokenów, odświeżania, błędów, integracji z dostawcami autentykacji i autoryzacji, zajmuje dużo czasu. Jednocześnie rozwiązania zastosowane w tej bibliotece pozostają elastyczne i możliwe do rozbudowy.

¹³Middleware – warstwa pośrednia oprogramowania działająca pomiędzy żądaniem użytkownika a właściwą logiką aplikacji, umożliwiająca przechwytywanie i modyfikację zapytań oraz odpowiedzi. Stosowana jest m.in. do realizacji takich zadań jak autentykacja, autoryzacja, logowanie zdarzeń, obsługa błędów czy przekierowania, bez ingerencji w główną logikę biznesową systemu.

¹⁴Por. [8].

4. Bazy Danych

Baza danych to system służący do trwałego przechowywania, organizowania oraz zarządzania danymi, wykorzystywanymi przez aplikację. Umożliwia ona zapisywanie informacji w ustrukturyzowanej formie, ich wyszukiwanie, modyfikację oraz usuwanie, zapewniając jednocześnie spójność, integralność i bezpieczeństwo przechowywanych informacji. Najczęściej stosuje się bazy relacyjne lub nierelacyjne, w zależności od charakteru przetwarzanych danych. Relacyjne bazy danych to systemy przechowujące dane w postaci tabel, składających się z wierszy i kolumn, powiązanych ze sobą za pomocą relacji. Wykorzystują one ustrukturyzowany schemat danych oraz język zapytań SQL (Structured Query Language), co zapewnia wysoką spójność i integralność informacji. Sprawdzają się szczególnie w aplikacjach, które wymagają precyzyjnych zależności między danymi. Bezrelacyjne bazy danych, zwane bazami noSQL, przechowują dane w bardziej elastycznej formie. Mogą to na przykład być dokumenty, pary klucz–wartość, grafy lub kolekcje. Takie bazy nie wymagają sztywno zdefiniowanego schematu, co ułatwia skalowanie oraz pracę z dużymi i zróżnicowanymi zbiorami danych. W architekturze aplikacji internetowej baza danych współpracuje przede wszystkim z warstwą backendową. Backend realizuje zapytania do bazy, przetwarza otrzymane wyniki zgodnie z logiką biznesową, a następnie udostępnia je frontendowi za pośrednictwem interfejsów API. Frontend odpowiada za prezentację danych, jednak nie komunikuje się bezpośrednio z bazą. Takie rozdzielenie odpowiedzialności zwiększa bezpieczeństwo systemu, ułatwia utrzymanie kodu oraz pozwala na skalowanie poszczególnych warstw aplikacji niezależnie od siebie.



Rys. 8. Przedstawienie procesu, umożliwiającego wyświetlenie danych użytkownikowi aplikacji internetowej. Źródło: [9].

4.1 MySQL

MySQL należy do najczęściej wykorzystywanych technologii bazodanowych w aplikacjach internetowych ze względu na wysoką wydajność, stabilność, łatwość konfiguracji oraz szerokie wsparcie społeczności i narzędzi programistycznych. Ten typ bazy danych wykorzystuje język SQL do definiowania struktury danych, wykonywania zapytań oraz zarządzania integralnością informacji. Dane w MySQL przechowywane są w tabelach złożonych z wierszy, tak zwanych rekordów i kolumn, zwanych atrybutami. Relacje pomiędzy tabelami realizowane są za pomocą kluczy głównych i obcych, co pozwala zachować spójność danych oraz minimalizować redundancję¹. Klucz główny to o kolumna lub zestaw kolumn, których wartość jednoznacznie identyfikuje każdy rekord w tabeli. Wartości klucza głównego muszą być unikalne i nie mogą przyjmować wartości pustych (wartości null). Natomiast klucz obcy to kolumna lub zestaw kolumn w tabeli, odwołujący się do klucza głównego w innej tabeli. Służy on do tworzenia powiązań między danymi, np. przypisania zamówienia do konkretnego użytkownika. Mechanizm kluczy obcych wymusza spójność relacyjną, zapobiegając wprowadzaniu odwołań do nieistniejących rekordów. MySQL obsługuje transakcje, indeksowanie oraz mechanizmy buforowania, co zapewnia bezpieczne i szybkie operacje odczytu oraz zapisu. Backend aplikacji komunikuje się z bazą poprzez zapytania, pobierając lub modyfikując dane, a następnie przekazuje je do warstwy frontendowej za pośrednictwem interfejsów API. Takie podejście oddziela logikę biznesową od warstwy prezentacji oraz zwiększa bezpieczeństwo systemu. Do kluczowych funkcjonalności MySQL należą:

- obsługa zapytań, takich jak SELECT, INSERT, UPDATE, DELETE, realizujących

¹Redundancja - nadmiarowość danych, polegająca na wielokrotnym przechowywaniu tych samych informacji w różnych miejscach systemu.

operacje na danych,

- transakcje - odpowiedzialne za wykonywanie kilku różnych operacji na bazie danych jednocześnie, układając je w jedną logiczną całość,
- indeksy - unikalnie identyfikują i przyspieszają wyszukiwanie rekordów,
- tworzenie widoków, czyli wirtualnych tabel, upraszczających złożone zapytania,
- procedury składowane i wyzwalacze, służące do automatyzowania operacji po stronie bazy danych,
- zarządzanie użytkownikami, uprawnieniami oraz szyfrowaniem połączeń.

Dzięki swojej niezawodności oraz dobrej integracji z popularnymi technologiami backendowymi MySQL stanowi uniwersalne i efektywne rozwiązanie dla aplikacji wymagających uporządkowanego modelu danych oraz wysokiej spójności informacji. System ten doskonale sprawdza się w aplikacjach administracyjnych, systemach rezerwacji, portalach społecznościowych, czy sklepach internetowych.

4.2 MongoDB

MongoDB² jest nierelacyjnym systemem zarządzania bazą danych, którego podstawową różnicą względem relacyjnych rozwiązań, jest sposób przechowywania informacji. MongoDB wykorzystuje model dokumentowy, w którym dane zapisywane są w elastycznych strukturach przypominających format JSON³. Oznacza to brak sztywno zdefiniowanego schematu tabel oraz większą swobodę w organizacji informacji. MongoDB przechowuje powiązane dane często w obrębie jednego dokumentu. Takie podejście redukuje potrzebę wykonywania złożonych połączeń między poszczególnymi tabelami, co przyspiesza operacje odczytu i upraszcza projektowanie aplikacji o dynamicznej strukturze danych. MongoDB zapisuje dane w kolekcjach składających się z dokumentów. Każdy dokument może posiadać inną strukturę oraz zestaw pól, co pozwala na łatwe rozszerzanie modelu danych bez konieczności modyfikowania schematu całej bazy. Komunikacja z bazą odbywa się poprzez zapytania realizowane w formie obiektów lub metod API, zamiast klasycznych zapytań w języku SQL. Do kluczowych możliwości MongoDB należą:

elastyczny schemat danych (schema-less),

przechowywanie danych w strukturze dokumentowej w formacie JSON lub BSON⁴,

wysoka skalowalność pozioma⁵,

²Por. [10]: *Welcome to the MongoDB docs*, <https://www.mongodb.com/docs/> (dostęp: 02.02.2026).

³JSON - z jęz. ang JavaScript Object Notation, tekstowy format zapisu i wymiany danych oparty na strukturze obiektów oraz tablic, wykorzystywany w komunikacji między aplikacjami.

⁴BSON - skrót od binary JSON, binarna reprezentacja formatu JSON.

⁵Skalowanie poziome - metoda zwiększania wydajności i pojemności systemu poprzez dodawanie kolejnych serwerów lub komputerów (zwanych węzłami), zamiast rozbudowy pojedynczej maszyny.

wydajne operacje odczytu i zapisu dla dużych zbiorów danych, zaawansowane mechanizmy filtrowania⁶ i agregacji⁷ danych.

MongoDB znajduje zastosowanie w projektach, w których dane mają zmienną strukturę lub bardzo dużą skalę. Jako przykłady można podać systemy zarządzania treścią, systemy analityczne, operujące na dużych ilościach danych lub aplikacje mobilne oraz internetowe, charakteryzujące się dynamicznie zmieniającymi się modelami danych. MySQL sprawdza się lepiej w systemach wymagających ścisłych relacji i spójności danych, natomiast MongoDB oferuje większą elastyczność i łatwiejsze skalowanie.

4.3 Uzasadnienie wyboru MongoDB

W projekcie aplikacji, będącej tematem niniejszej pracy, kluczowe znaczenie ma łatwość rozbudowy funkcjonalności oraz możliwość obsługi dynamicznie zmieniających się struktur informacji. Z tego względu zdecydowano się na zastosowanie bazy dokumentowej MongoDB zamiast klasycznej relacyjnej bazy danych. W systemie zarządzania zadaniami pojedyncze obiekty, takie jak projekt lub zadanie, mogą posiadać zróżnicowane właściwości. Model dokumentowy MongoDB pozwala przechowywać takie dane w postaci jednego, zagnieźdzonego dokumentu, bez konieczności tworzenia wielu tabel i relacji. Ułatwia to modyfikację struktury danych w trakcie rozwoju aplikacji, bez potrzeby przeprowadzania kosztownych migracji schematu. Dodatkową zaletą jest wysoka wydajność operacji odczytu i zapisu oraz możliwość łatwego skalowania poziomego, co ma istotne znaczenie w przypadku rosnącej liczby użytkowników i zadań. Brak konieczności wykonywania złożonych połączeń upraszcza zapytania oraz przyspiesza dostęp do danych, szczególnie gdy informacje dotyczące jednego projektu są przechowywane w jednym dokumencie. Ponadto MongoDB znakomicie integruje się z technologiami JavaScript, co sprzyja spójności technologicznej projektu i upraszcza proces programowania. NextJS, będący częścią stosu technologicznego projektu aplikacji, zapewnia dobre wsparcie dla narzędzi oferowanych przez MongoDB.

⁶Filtrowanie - operacja na danych, polegająca na selekcji rekordów spełniających określone warunki

⁷Agregacja - operacja na danych, polegająca na łączeniu i podsumowywaniu danych w celu uzyskania zbiorczych wyników, takich jak sumy, średnie lub grupowanie wyników.

5. Architektura aplikacji

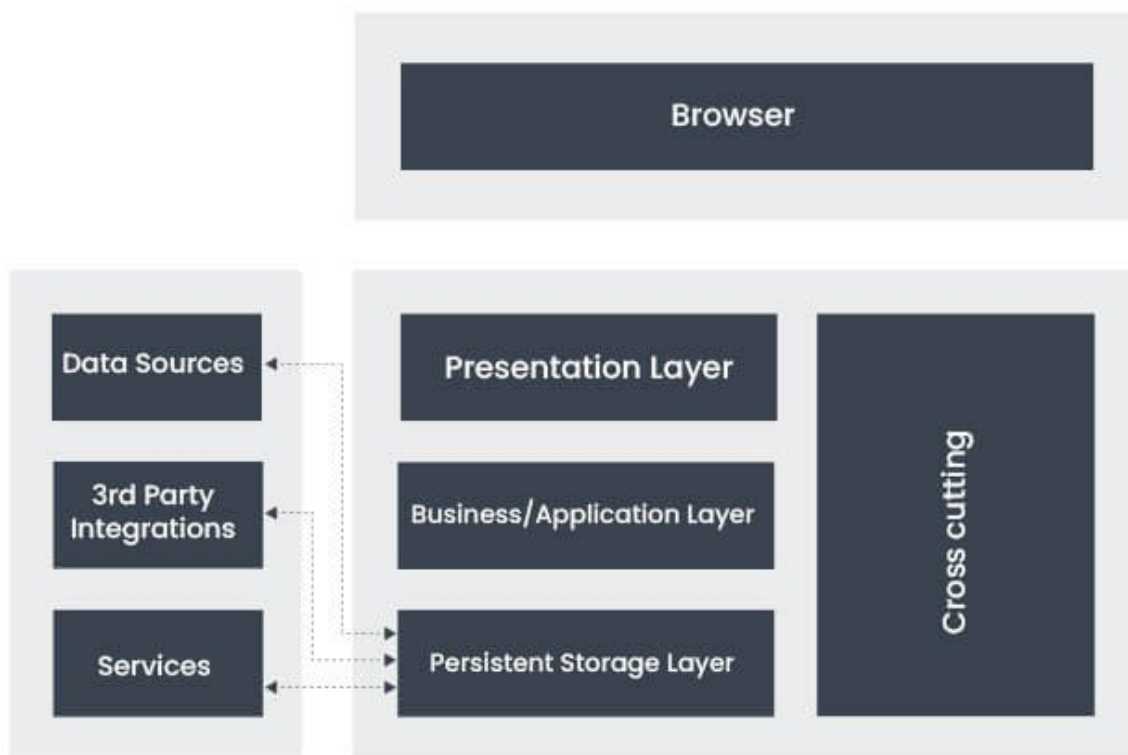
Architektura klient-serwer jest jednym z podstawowych modeli projektowania aplikacji internetowych, w którym system podzielony jest na dwie warstwy - klienta oraz serwera. Klient odpowiada za interakcję z użytkownikiem i prezentację danych, natomiast serwer realizuje logikę biznesową, przetwarzanie danych oraz komunikację z bazą danych. W tym modelu komunikacja odbywa się w schemacie “żądanie - odpowiedź”. Klient wysyła do serwera zapytanie, np. o pobranie danych lub zapis informacji, a serwer przetwarza je i zwraca odpowiedź w postaci danych lub komunikatu o wyniku operacji. Takie rozdzielenie odpowiedzialności pozwala oddzielić warstwę prezentacji od warstwy przetwarzania danych, co zwiększa przejrzystość projektu oraz ułatwia jego rozwój. Istotną zaletą architektury klient-serwer jest możliwość niezależnego skalowania poszczególnych części systemu. Warstwa serwerowa może być rozbudowywana w celu obsługi większej liczby użytkowników, natomiast frontend może być rozwijany niezależnie od zmian w logice backendu. Dodatkowo centralizacja przetwarzania danych po stronie serwera zwiększa bezpieczeństwo, ponieważ użytkownik nie ma bezpośredniego dostępu do bazy danych ani wewnętrznych mechanizmów aplikacji. Architektura ta stanowi fundament współczesnych aplikacji internetowych, umożliwiając tworzenie systemów modularnych, wydajnych i łatwych w utrzymaniu, co ma szczególne znaczenie w projektach o rosnącej złożoności i liczbie użytkowników. Rozpatrzmy funkcjonowanie architektury klient-serwer w przypadku projektu aplikacji do organizowania zadań. Warstwa klienta, realizowana w przeglądarce internetowej, odpowiada za prezentację interfejsu użytkownika, obsługę interakcji oraz wysyłanie żądań dotyczących operacji na danych, takich jak dodawanie, edycja czy usuwanie zadań. Warstwa serwerowa przetwarza te żądania, realizuje logikę biznesową aplikacji, zarządza autentykacją użytkowników oraz komunikuje się z bazą danych, w której przechowywane są informacje o projektach i zadaniach. W systemie wymagającym logowania użytkownika, szczególne znaczenie musi mieć bezpieczeństwo. Dzięki rozdzieleniu warstw, klient nie posiada bezpośredniego dostępu do bazy danych, ani wrażliwych mechanizmów aplikacji. Wszelkie operacje wykonywane są po stronie serwera, gdzie możliwa jest weryfikacja tożsamości użytkownika, kontrola uprawnień oraz walidacja przesyłanych danych. Pozwala to zapobiegać nieautoryzowanemu dostępowi do cudzych zasobów, a także ogranicza ryzyko manipulacji danymi lub ataków polegających na modyfikacji kodu po stronie przeglądarki. Zmniejsza to również ryzyko wycieku danych poufnych i dostania się ich w niepowołane ręce.

5.1 Omówienie warstw aplikacji

Aplikację internetową można zasadniczo podzielić na trzy warstwy:

- Prezentacji (frontend),
- Logiki aplikacji (backend),
- Danych.

Na poniżej przedstawionym schemacie zaprezentowano klasyczną, trójwarstwową architekturę aplikacji internetowej. Poszczególne warstwy odpowiadają za różne obszary odpowiedzialności systemu, co zwiększa modularność, bezpieczeństwo oraz łatwość utrzymania oprogramowania.



Rys. 9. Diagram, przedstawiający warstwy aplikacji internetowej. Źródło: [11].

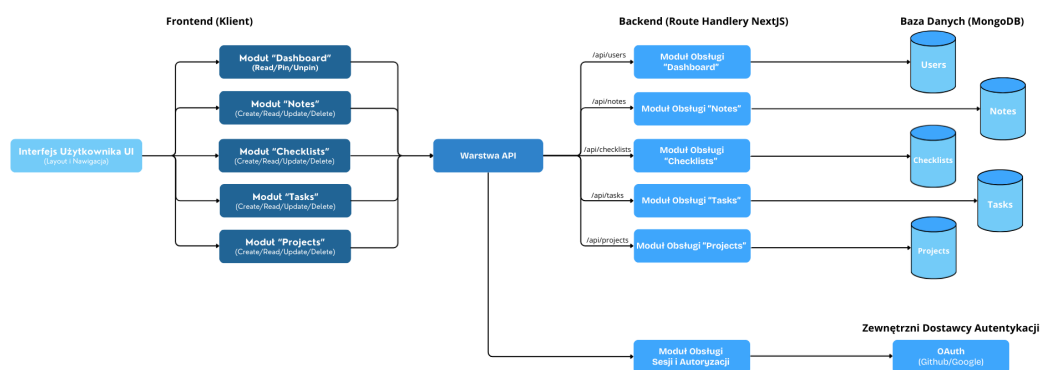
Przeglądarka (browser) to środowisko uruchomieniowe po stronie klienta, w którym działa aplikacja frontendowa. Użytkownik korzysta z systemu poprzez przeglądarkę, która renderuje interfejs i wysyła zapytania do serwera. Ponadto umożliwia wykonywanie kodu JavaScripta, obsługuje interakcje z systemem, wyświetla interfejs użytkownika oraz może komunikować się z backendem za pomocą protokołów HTTP lub HTTPS¹. Warstwa prezentacji odpowiada za interfejs użytkownika (UI) oraz sposób prezentacji danych. Wyświetla

¹HTTP (HyperText Transfer Protocol) – protokół komunikacyjny warstwy aplikacji wykorzystywany do przesyłania danych pomiędzy klientem a serwerem w sieci WWW. Jego rozszerzona, bardziej bezpieczna wersja, wykorzystująca mechanizmy szyfrowania to protokół HTTPS (HyperText Transfer Protocol Secure).

widoki formularzy, zbiera dane wejściowe, które potem mogą zostać przesłane w postaci zapytań do bazy danych oraz zajmuje się podstawową walidacją formularzy. Odpowiada również za wysyłanie zapytań do backendu. Warstwa logiki biznesowej stanowi rdzeń aplikacji, odpowiadając za przetwarzanie danych. Realizuje operacje CRUD, dbając też o szczegółową walidację danych. Realizuje autentykację i autoryzację użytkowników. Za magazynowanie danych w sposób trwały odpowiada warstwa przechowywania danych (na rysunku opisana jako persistent storage layer), będąca zazwyczaj bazą danych. Ta część aplikacji odczytuje, przechowuje i zapewnia integralność danych. Źródła zewnętrzne z kolei to zewnętrzne systemy i usługi, z którymi aplikacja się integruje. Mogą to być zewnętrzne bazy lub pliki z danymi (data sources), integracje z systemami firm trzecich (3rd party integrations) lub dodatkowe mikroserwisy oraz API (na schemacie opisane jako services). Mechanizmy przekrojowe (na schemacie, widniejące jako cross-cutting) to elementy, działające na wszystkich warstwach. Nie należą one do konkretnej warstwy, lecz wpływają na cały system. Mogą obejmować na przykład mechanizmy bezpieczeństwa, logowania zdarzeń, obsługę błędów lub monitorowanie wydajności aplikacji.

5.2 Diagramy Architektoniczne

Jednym z najważniejszych diagramów, stworzonych podczas procesu projektowania fundamentów aplikacji jest diagram komponentów. Przedstawia on architekturę komponentową aplikacji zrealizowaną w modelu klient-serwer, z wyraźnym podziałem na interfejs użytkownika (frontend), warstwę API, logikę serwerową (backend), bazę danych oraz zewnętrznych dostawców autentykacji. Strzałki obrazują zależności oraz przepływ danych między komponentami.



Rys. 10. Prosty diagram komponentów internetowej aplikacji do zarządzania zadaniami.

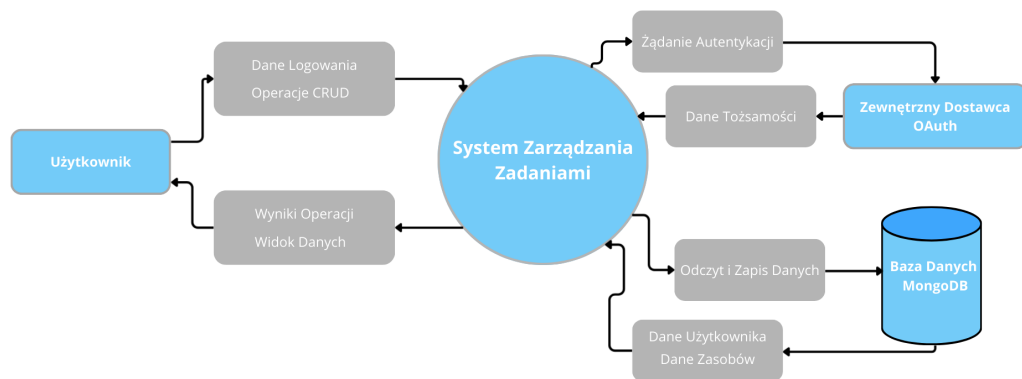
Przepływ danych można rozpisać na poszczególne kroki:

1. Użytkownik wykonuje akcję w interfejsie.
2. Frontend wysyła żądanie do API.
3. Backend przetwarza logikę, sprawdzając również uprawnienia.
4. Dane są pobierane lub zapisywane w bazie.
5. Odpowiedź wraca do UI.

Najważniejsze idee architektoniczne:

- separacja warstw,
- modularność, wynikająca z osobnych serwisów dla każdego typu zasobu,
- bezpieczeństwo, zapewnione przez brak bezpośredniego dostępu klienta do bazy,
- łatwa skalowalność.

Kolejne diagramy, powstałe na etapie projektowania architektury przygotowano, wykorzystując informacje z artykułu “Canva: DFD”². Pokazują jak dane przemieszczają się w systemie między źródłami, procesami oraz magazynami danych. Są to tak zwane jest “diagramy przepływu danych”.

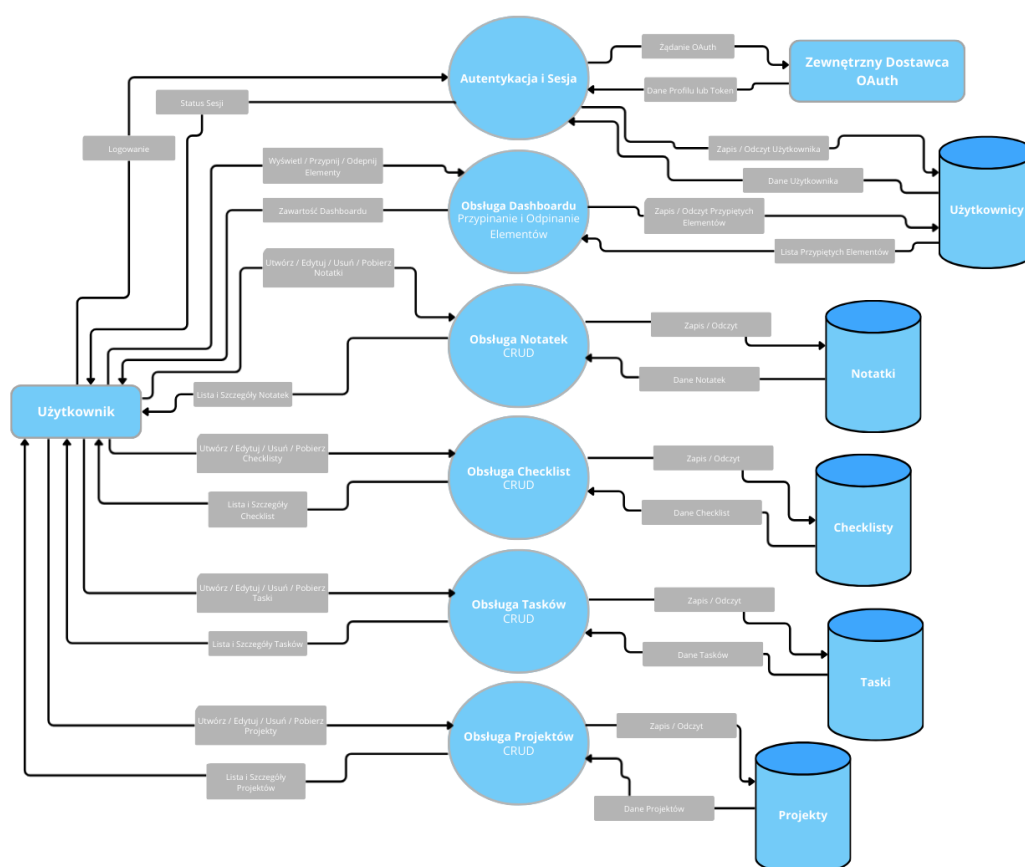


Rys. 11. Diagram przepływu danych poziomy 0, przedstawiający system jako jeden proces wraz z relacjami z otoczeniem.

²Por. [12]: Canva Team, *Creating a data flow diagram: How-tos, templates, and tips*, <https://www.canva.com/online-whiteboard/data-flow-diagrams/> (dostęp: 03.02.2026).

Diagram przepływu danych poziomu 0 przedstawia aplikację do zarządzania zadaniami jako pojedynczy, logiczny proces, przetwarzający dane pomiędzy użytkownikiem, zewnętrznym dostawcą autentykacji oraz bazą danych. Użytkownik inicjuje operacje takie jak logowanie oraz zarządzanie zadaniami, projektami, notatkami i checklistami, natomiast system przetwarza te żądania, komunikuje się z bazą danych oraz zwraca odpowiednie wyniki. Diagram ten obrazuje granice systemu oraz jego relacje z otoczeniem, pomijając szczegóły wewnętrznej struktury aplikacji, które przedstawiane są na kolejnym rysunku.

Diagram przepływu danych poziomu 1 przedstawia szczegółową dekompozycję systemu na główne procesy funkcjonalne oraz pokazuje sposób wymiany informacji pomiędzy użytkownikiem, modułami aplikacji, bazą danych i zewnętrznym dostawcą autentykacji. Użytkownik inicjuje operacje takie jak logowanie, przeglądanie oraz zarządzanie zadaniami, projektami, notatkami i checklistami, a odpowiednie procesy systemowe realizują logikę biznesową i wykonują operacje zapisu lub odczytu danych w dedykowanych kolekcjach bazy danych. Dodatkowo wydzielony został proces autentykacji i zarządzania sesją, który komunikuje się z dostawcą OAuth w celu weryfikacji tożsamości użytkownika. Diagram ilustruje rozdzielenie odpowiedzialności pomiędzy warstwą logiki aplikacji i warstwą przechowywania danych.



Rys. 12. Diagram przepływu danych poziomu 1

5.3 Struktura bazy danych aplikacji

W aplikacji zastosowano bazę danych MongoDB oraz bibliotekę Mongoose, która została wykorzystana do opisanie struktury dokumentów domenowych. Połączenie z bazą jest realizowane przez moduł odpowiedzialny za utrzymywanie współdzielonego połączenia Mongoose, natomiast mechanizm logowania OAuth korzysta dodatkowo z adaptera MongoDB dla NextAuth. Dzięki temu dane logowania oraz dane aplikacyjne są przechowywane w tej samej bazie, ale obsługiwane przez odrębne warstwy aplikacji.

Podstawową zasadą modelu danych jest przypisanie dokumentów do konkretnego użytkownika. Notatki, checklistsy, zadania, projekty, przypięcia oraz zdarzenia aktywności zawierają pole `ownerId`, które wskazuje właściciela dokumentu. Preferencje interfejsu użytkownika korzystają z pola `userId`. Każda operacja pobierania, aktualizacji lub archiwizacji danych wykonywana jest z uwzględnieniem identyfikatora aktualnie zalogowanego użytkownika, co zabezpiecza aplikację przed dostępem do cudzych zasobów.

Warstwa autoryzacji tworzy kolekcje `users`, `accounts`, `sessions` oraz `verification_tokens`. Najważniejsze z punktu widzenia działania aplikacji są kolekcje `users` i `accounts`, ponieważ przechowują użytkowników oraz powiązania z zewnętrznymi dostawcami logowania, takimi jak Google i GitHub. Sesja użytkownika jest obsługiwana z użyciem strategii JWT, jednak identyfikator użytkownika pochodzący z NextAuth stanowi podstawę powiązań z dokumentami domenowymi.

Główne dane aplikacji zostały podzielone na kolekcje `notes`, `checklists`, `tasks`, `projects`, `pins`, `userpreferences` oraz `activityevents`. Kolekcja `notes` przechowuje notatki użytkownika, zawierające tytuł, treść, tagi, pozycję sortowania oraz listę powiązanych elementów. Powiązania notatki są przechowywane w tablicy `linkedItems`, gdzie każdy wpis składa się z typu obiektu oraz jego identyfikatora. Dzięki temu notatka może odnosić się do innej notatki, checklisty, zadania albo projektu.

Checklisty są przechowywane w kolekcji `checklists`. Każda checklista posiada tytuł, pozycję, opcjonalne powiązanie z rodzicem oraz tablicę elementów. Elementy checklisty zostały osadzone bezpośrednio w dokumencie checklisty, ponieważ są odczytywane i aktualizowane razem z nią. Pojedynczy element zawiera nazwę, informację o ukończeniu, datę ukończenia oraz pozycję. Checklista może funkcjonować samodzielnie albo być powiązana z zadaniem lub projektem przez pola `parentType` i `parentId`.

Zadania są zapisywane w kolekcji `tasks`. Dokument zadania zawiera tytuł, opis, priorytet, status, termin wykonania, tagi, pozycję, datę ukończenia oraz datę archiwizacji. Zadanie może być samodzielnym elementem listy zadań albo częścią projektu. W drugim przypadku pole `projectId` wskazuje projekt, do którego należy zadanie. Status zadania jest przechowywany w polu `statusId`; w widoku tablicy kanban odpowiada on identyfikatorowi jednej z kolumn projektu. Zadanie może mieć również powiązane checklisty, których identyfikatory są przechowywane w tablicy `checklistIds`.

Projekty są przechowywane w kolekcji `projects`. Dokument projektu zawiera tytuł, opis, priorytet, status cyklu życia, termin, tagi, pozycję, listę zadań, checklisty oraz konfigurację

widoku zadań. W projekcie przechowywana jest tablica `kanbanColumns`, opisująca kolumny tablicy kanban. Każda kolumna ma identyfikator, nazwę, kolor, pozycję oraz informację, czy oznacza stan zakończony. Domyślny zestaw kolumn obejmuje backlog, todo, `in_progress`, testing oraz done. Pole `taskView` określa, czy zadania projektu są prezentowane jako lista, czy jako tablica kanban.

Relacja pomiędzy projektami i zadaniami jest utrzymywana dwukierunkowo na poziomie aplikacji. Główne powiązanie znajduje się w zadaniu, w polu `projectId`. Projekt przechowuje dodatkowo tablicę `taskIds`, która zawiera identyfikatory powiązanych zadań. Podczas tworzenia zadania z przypisanym projektem backend sprawdza, czy projekt istnieje, nie jest zarchiwizowany i należy do aktualnego użytkownika. Następnie identyfikator zadania jest dopisywany do projektu. Przy zmianie projektu lub archiwizacji zadania identyfikator jest usuwany ze starego projektu.

Kolekcja `pins` przechowuje elementy przypięte na ekranie głównym aplikacji. Dokument przypięcia zawiera typ wskazywanego obiektu, jego identyfikator oraz pozycję na dashboardzie. Dzięki parze pól `targetType` i `targetId` możliwe jest przypięcie notatki, checklisty, zadania albo projektu bez tworzenia osobnych kolekcji dla każdego typu przypięcia. W modelu zastosowano unikalny indeks obejmujący właściciela, typ celu i identyfikator celu, co zapobiega wielokrotnemu przypięciu tego samego elementu przez jednego użytkownika.

Preferencje interfejsu zapisano w kolekcji `userpreferences`. Dokument preferencji zawiera tryb kolorystyczny, układ dashboardu, zestaw kolorów używanych w aplikacji oraz zapisane własne motywy. Pole `userId` jest unikalne, dlatego jeden użytkownik ma jeden dokument preferencji. Takie rozwiązanie upraszcza pobieranie ustawień podczas renderowania interfejsu.

Kolekcja `activityevents` przechowuje zdarzenia aktywności. Zdarzenie zawiera typ encji, identyfikator dokumentu, rodzaj akcji oraz czas wystąpienia. Rejestrowane są między innymi operacje utworzenia, aktualizacji, usunięcia, przeniesienia oraz przypięcia elementu. Endpoint odpowiedzialny za komunikację w czasie rzeczywistym odczytuje te zdarzenia i przekazuje je klientowi za pomocą mechanizmu SSE, co pozwala odświeżać widoki po zmianach danych.

Najważniejsze relacje logiczne w bazie danych można przedstawić następująco:

- użytkownik jest właścicielem notatek, checklist, zadań, projektów, przypięć i zdarzeń aktywności,
- użytkownik posiada jeden dokument preferencji interfejsu,
- zadanie może należeć do projektu przez pole `projectId`,
- projekt przechowuje pomocniczą listę zadań w tablicy `taskIds`,
- zadanie i projekt mogą mieć powiązane checklisty,
- notatka może wskazywać inne elementy aplikacji przez tablicę `linkedItems`,

- przypięcie wskazuje dowolny obsługiwany typ elementu przez pola targetType i targetId.

W bazie zastosowano indeksy wspierające najczęstsze operacje aplikacji. Większość kolekcji posiada indeks po polu właściciela, co przyspiesza pobieranie danych aktualnego użytkownika. Dodatkowe indeksy obejmują między innymi pozycję sortowania, tagi, termin wykonania, priorytet, status zadania oraz pola wykorzystywane do przypięć. Dzięki temu aplikacja może sprawnie filtrować zadania i projekty, sortować elementy dashboardu oraz szybko sprawdzać, czy dany element został już przypięty.

Usuwanie większości elementów domenowych zostało zrealizowane jako archiwizacja. Zadania, projekty i checklisty otrzymują wartość w polu archivedAt, a zapytania listujące pobierają wyłącznie dokumenty aktywne. W przypadku projektu podczas archiwizacji ustalany jest również status cyklu życia archived. Takie podejście pozwala zachować spójność powiązań i historię zmian bez fizycznego usuwania dokumentów. Model dokumentowy MongoDB jest w tym projekcie uzasadniony, ponieważ dane mają strukturę elastyczną, a część informacji, takich jak elementy checklisty i kolumny kanban, jest naturalnie powiązana z dokumentem nadrzędnym i może być przechowywana bezpośrednio w jego strukturze.

6. Projekt interfejsu użytkownika

Jak już wcześniej ustalono, projekt interfejsu użytkownika, jest jednym z najważniejszych aspektów aplikacji. Przejrzystość, czytelność oraz prostota obsługi decydują w dużej mierze o użyteczności niniejszej aplikacji. W tym celu najpierw rozpisane zostały historie użytkownika, po których analizie powstał graficzny projekt interfejsu.

6.1 Scenariusze przypadków użycia

1) Przypięcie elementu do strony głównej

Aktor: Użytkownik indywidualny

Cel: Szybki dostęp do śledzonych tasków/notatek/checklist/projektów z poziomu strony głównej

Warunki wstępne: Użytkownik jest zalogowany; Element istnieje

Scenariusz podstawowy:

1. Użytkownik otwiera którąś z list elementów (taski/checklisty/projekty itd.)
2. Użytkownik wybiera konkretny element
3. Użytkownik widzi, że element nie jest przypięty (ikonka pinezki pusta w środku)
4. Użytkownik klika na ikonkę i dodaje do przypiętych
5. System oznacza element jako „przypięty” poprzez zmianę ikonki pinezki na wypełnioną
6. System wyświetla element w sekcji „przypięte” na stronie głównej.

Scenariusze alternatywne / wyjątki:

1. Element już jest przypięty

- 3a. System daje użytkownikowi informację, że element jest już przypięty poprzez (wypełnioną ikonę pinezki)
- 4a. Użytkownik może kliknąć ikonkę w celu odpięcia elementu od ekranu głównego.

Warunki końcowe: Element jest widoczny w sekcji „przypięte” na stronie głównej

2) Utworzenie nowego zadania z parametrami

Aktor: Pracownik biurowy

Cel: Uporządkowanie pracy poprzez utworzenie zadania z właściwościami

Warunki wstępne: Użytkownik jest zalogowany

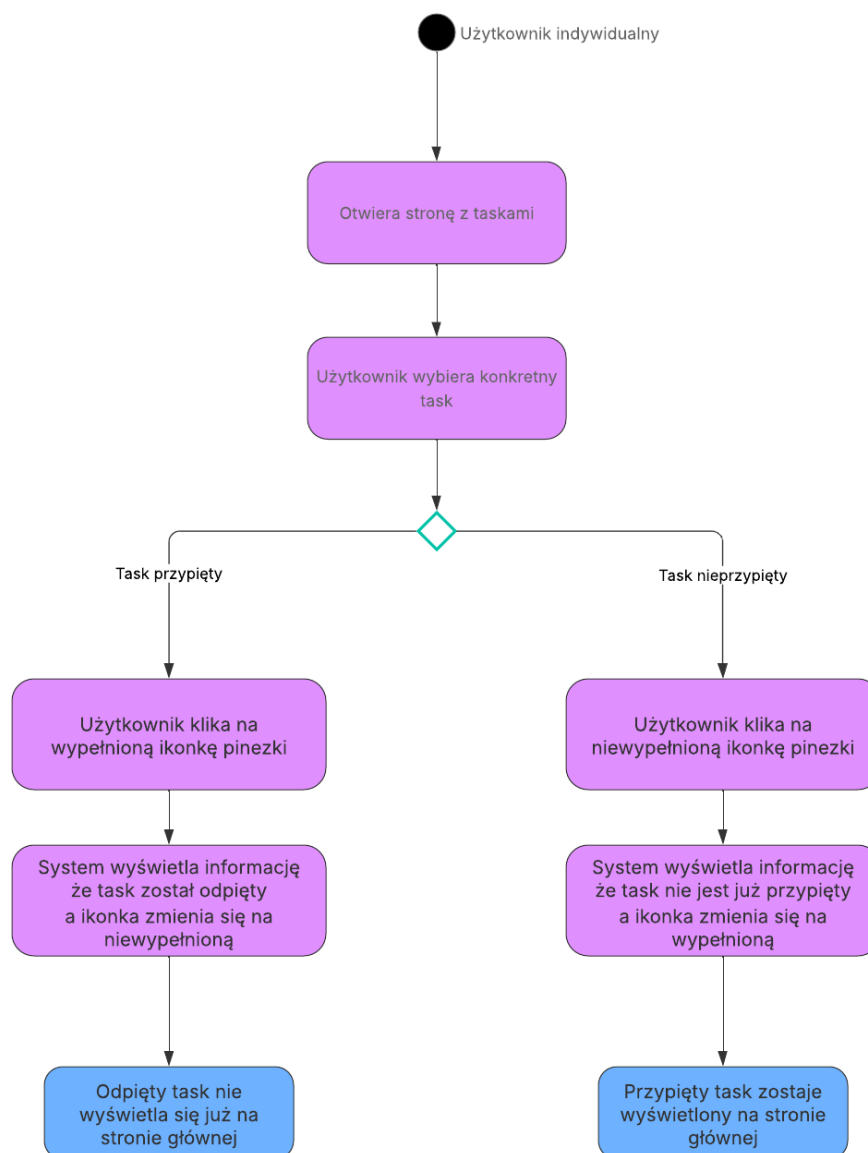
Scenariusz podstawowy:

1. Użytkownik wybiera z paska bocznego zakładkę “zadania”
2. Użytkownik zostaje przekierowany na stronę z zadaniami
3. Użytkownik wybiera ikonkę z plusem, znajdującą się obok istniejących zadań
4. System otwiera formularz tworzenia zadania
5. Użytkownik wpisuje nazwę zadania
6. Użytkownik uzupełnia opis zadania
7. Użytkownik wybiera priorytet z rozwijanej listy wyboru
8. Użytkownik ustawia datę realizacji zadania
9. Użytkownik zapisuje zadanie
10. System tworzy zadanie i wyświetla je w liście zadań na stronie

Scenariusze alternatywne / wyjątki:

1. **Brak opisu**
 - 6a. Użytkownik pomija opis; system zapisuje zadanie bez opisu.
2. **Brak wymaganych danych (np. nazwy)**
 - 5a. Użytkownik pomija nazwę zadania.
 - 9a. System blokuje zapis i wskazuje brakujące pola.
3. **Niepoprawna data realizacji**
 - 8a. Użytkownik ustawia datę z przeszłości.
 - 9a. System blokuje zapis i wskazuje błędną datę.

Warunki końcowe: Dodane zadanie wraz z opisem, priorytetem i datą realizacji pojawiają się w liście zadań



Rys. 13. *Diagram UML, wizualizujący scenariusz przypięcia taska*

3) Utworzenie checklisty zakupów z możliwością odhaczanie pozycji

Aktor: Głowa rodziny

Cel: Nie zapomnieć produktów w sklepie

Warunki wstępne: Użytkownik jest zalogowany

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony “szybkie notatki”
2. Użytkownik wybiera ikonkę z plusem, znajdującą się obok istniejących elementów
3. Użytkownik wybiera czy chce dodać notatkę, czy checklistę
4. System otwiera formularz tworzenia checklisty

5. Użytkownik ustawia nazwę checklisty
6. Użytkownik dodaje pozycje w checkliście (np. mleko, chleb, warzywa)
7. System zapisuje checklistę i wyświetla ją użytkownikowi.
8. W sklepie użytkownik otwiera checklistę
9. Użytkownik odhacza kolejne pozycje
10. System zapisuje, które przedmioty w checkliście zostały odhaczone

Scenariusze alternatywne / wyjątki:

1. Dodanie pozycji w trakcie zakupów

- 7a. Użytkownik dodaje nową pozycję.
- 8a. System aktualizuje checklistę.

Warunki końcowe: Checklista zakupów istnieje, a pozycje mogą być odhaczane i zapisywane.

4) Tworzenie notatki

Aktor: Pracownik nadzorujący dostawy w restauracji

Cel: Utworzenie notatki zawierającej informacje o przebiegu dostawy oraz liczbie palet zwróconych przez restaurację.

Warunki wstępne: Użytkownik jest zalogowany w systemie; użytkownik ma dostęp do sekcji notatek.

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony z notatkami.
2. System wyświetla listę istniejących notatek.
3. Użytkownik wybiera przycisk dodania nowej notatki.
4. System otwiera formularz tworzenia notatki.
5. Użytkownik wpisuje tytuł notatki, np. „Dostawa - restauracja”.
6. Użytkownik wpisuje treść notatki, zawierającą informacje o przebiegu dostawy.
7. Użytkownik dopisuje informację o liczbie palet zwróconych przez restaurację.
8. Użytkownik opcjonalnie dodaje tagi porządkujące notatkę, np. „dostawa” lub „restauracja”.
9. Użytkownik zapisuje notatkę.
10. System tworzy notatkę i zapisuje ją w bazie danych.

11. System wyświetla utworzoną notatkę na liście notatek.

Scenariusze alternatywne / wyjątki:

1. **Brak treści notatki**

- 6a. Użytkownik pomija wpisanie treści notatki.
- 9a. System zapisuje notatkę z samym tytułem.

2. **Brak wymaganej nazwy notatki**

- 5a. Użytkownik nie wpisuje tytułu notatki.
- 9a. System blokuje zapis i informuje użytkownika o konieczności podania tytułu.

3. **Dodanie powiązania z innym elementem**

- 8a. Użytkownik wybiera element aplikacji, z którym notatka ma zostać powiązana, np. task, projekt lub checklistę.
- 9a. System zapisuje notatkę wraz z informacją o powiązanym elemencie.

4. **Anulowanie tworzenia notatki**

- 4a. System wyświetla formularz tworzenia notatki.
- 5a. Użytkownik rezygnuje z dodawania notatki i wraca do listy notatek.
- 6a. System nie zapisuje żadnych danych i ponownie wyświetla listę notatek.

Warunki końcowe: Notatka zostaje utworzona i jest widoczna na liście notatek. Użytkownik może później otworzyć jej podgląd, edytować treść, dodać tagi lub powiązać ją z innymi elementami aplikacji.

5) Utworzenie projektu i zarządzanie podzadaniami w widoku tablicy

Aktor: Student

Cel: Organizacja pisania pracy dyplomowej poprzez projekt i podzadania

Warunki wstępne: Użytkownik zalogowany

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony z projektami
2. Użytkownik wybiera ikonkę z plusem, znajdującą się obok istniejących projektów
3. System Otwiera formularz kreacji projektu
4. Użytkownik ustawia nazwę projektu
5. Użytkownik ustawia datę realizacji projektu
6. Użytkownik dodaje opis projektu
7. Użytkownik chce dodać taski do projektu
8. System otwiera użytkownikowi formularz dodawania tasków

9. Użytkownik może wybrać - czy dodać istniejący już task, czy stworzyć nowy
10. Użytkownik dodaje kilka tasków do projektu
11. Użytkownik ustawia statusy poszczególnym zadaniom
12. Użytkownik zapisuje projekt
13. Użytkownik uruchamia stronę projektu i przegląda jego opis
14. Użytkownik przełącza widok tasków z listy na tablicę
15. System wyświetla kolumny statusów oraz karty podzadań.
16. Użytkownik przeciąga karty między kolumnami, zmieniając w ten sposób ich status
17. System zapisuje zmiany statusów.

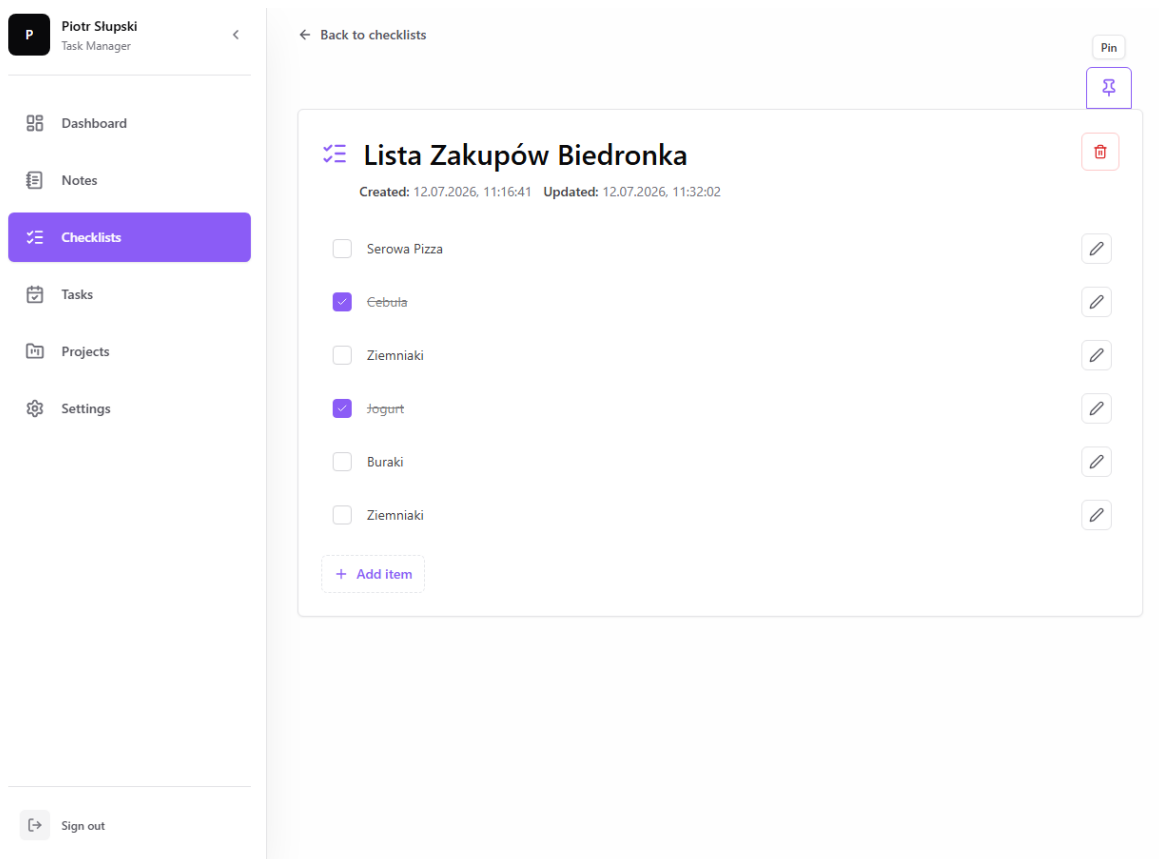
Scenariusze alternatywne / wyjątki:

1. **Brak daty realizacji**
 - 6a. Użytkownik pomija wypełnianie daty realizacji.
 - 12a. System zapisuje projekt bez daty realizacji.
2. **Brak wymaganych danych (np. nazwy)**
 - 4a. Użytkownik pomija nazwę projektu.
 - 12a. System blokuje zapis i wskazuje brakujące pola.
3. **Niepoprawna data realizacji**
 - 5a. Użytkownik ustawia datę z przeszłości.
 - 12a. System blokuje zapis i wskazuje błędną datę.

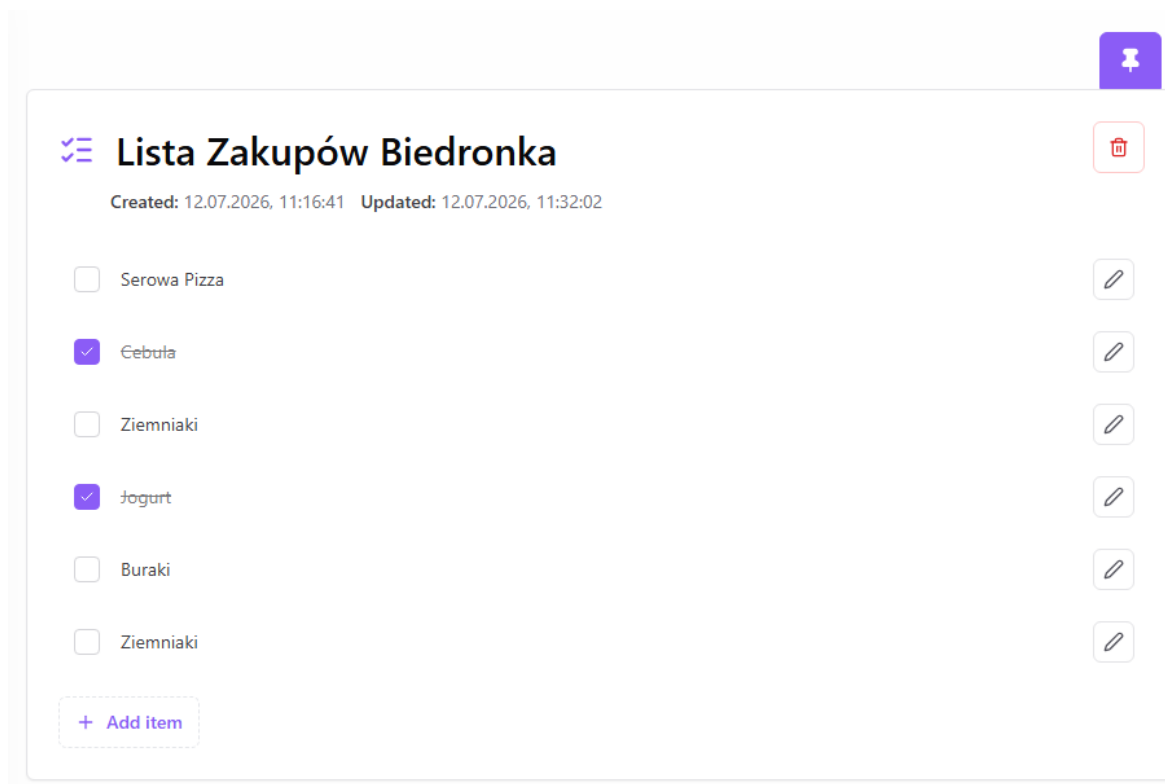
Warunki końcowe: Projekt istnieje z opisem, datą i innymi wypełnionymi parametrami a system może wyświetlać taski zarówno w formie listy jak i tablicy

6.2 Widoki użytkownika aplikacji

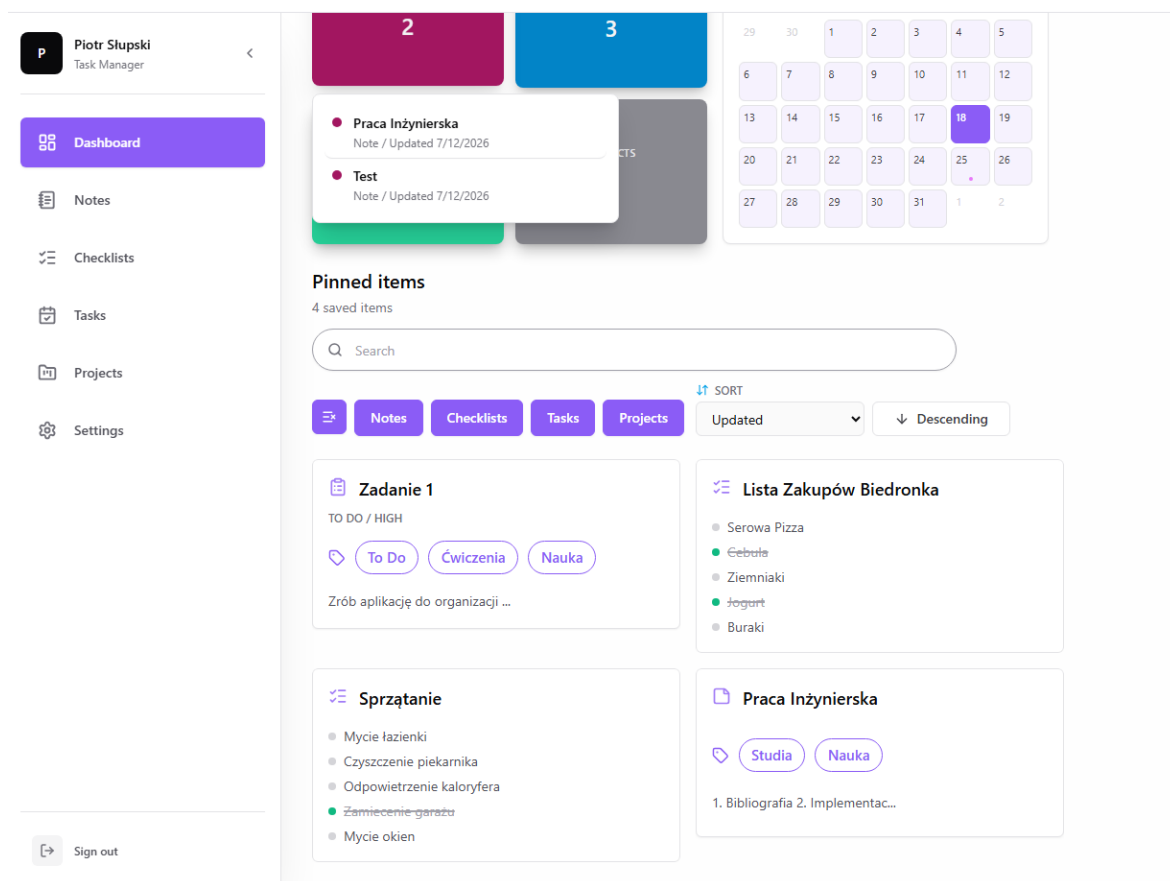
W niniejszym podrozdziale przedstawiono wybrane widoki aplikacji, które odpowiadają scenariuszom przypadków użycia opisanym w poprzedniej części rozdziału. Zaprezentowane ekrany dotyczą przypięcia checklisty do strony głównej, utworzenia taska oraz utworzenia projektu z taskami wyświetlanymi w widoku tablicy kanban i w widoku listy. Opis koncentruje się na układzie interfejsu graficznego, sposobie prezentacji danych oraz elementach ułatwiających użytkownikowi rozpoznanie aktualnego stanu wykonywanych czynności.



Rys. 14. Widok, przedstawiający checklistę, z możliwością przypięcia jej do ekranu głównego.

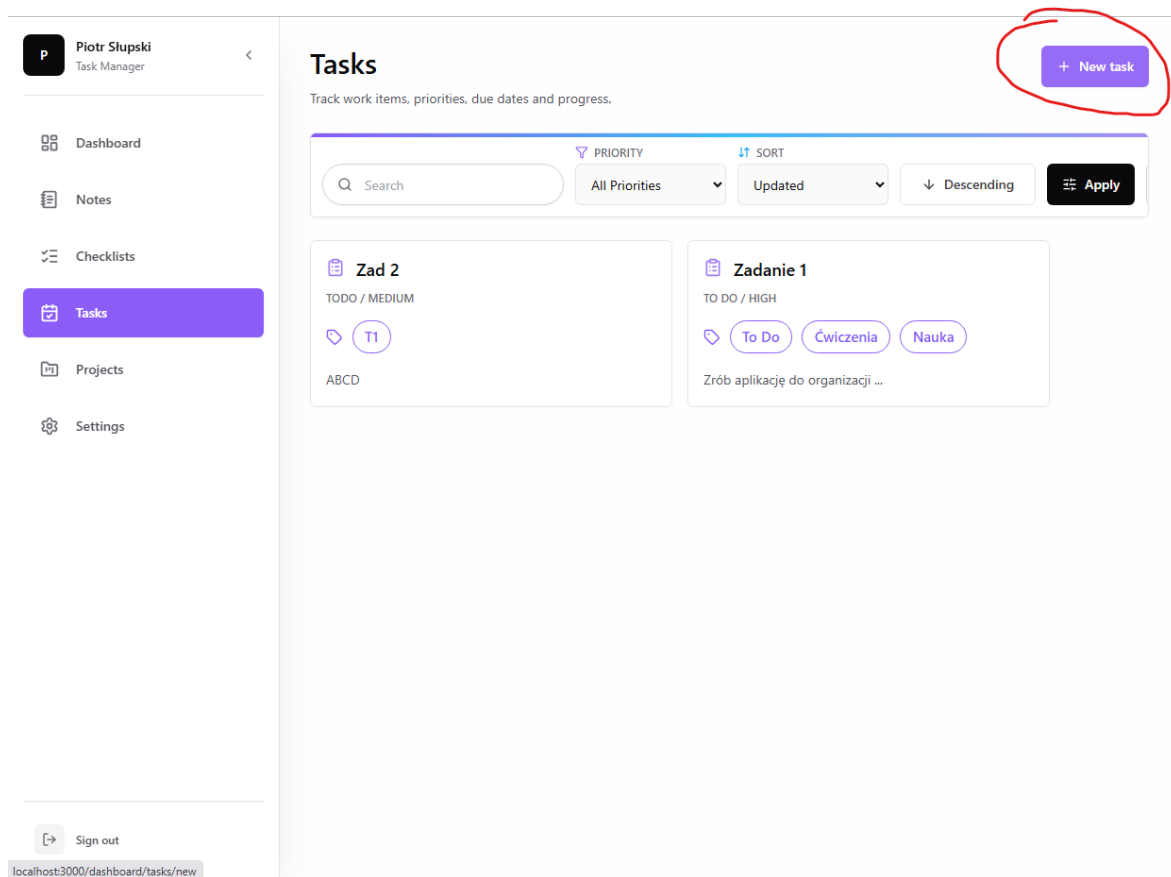


Rys. 15. Widok przedstawiający zachowanie przycisku “Przypnij” po przypięciu checklisty do ekranu głównego.



Rys. 16. Widok przedstawiający elementy przypięte do ekranu głównego.

Rysunki 14–16 przedstawiają przebieg scenariusza przypięcia checklisty do ekranu głównego. Widok szczegółów checklisty został zaprojektowany jako centralny panel z wyraźnym tytułem, ikoną typu obiektu oraz listą pozycji oznaczanych kwadratowymi polami wyboru. Wykonane elementy checklisty są odróżnione kolorem akcentu i przekreśleniem tekstu, dzięki czemu użytkownik może szybko ocenić postęp realizacji. Przycisk przypięcia został umieszczony przy prawej krawędzi panelu, w pobliżu pozostałych akcji na obiekcie. Po przypięciu zmienia się jego wypełnienie oraz kolorystyka, co daje czytelną informację zwrotną bez konieczności opuszczania aktualnego ekranu. Na dashboardzie przypięte elementy są prezentowane jako karty z ikonami typów, tytułami, tagami i krótkim podglądem zawartości, a ich rozmieszczenie w siatce pozwala skanować najważniejsze obiekty bez przechodzenia do osobnych modułów aplikacji.



Rys. 17. Widok, przedstawiający stronę z taskami użytkownika.

P Piotr Słupski
Task Manager

Dashboard

Notes

Checklists

Tasks

Projects

Settings

Sign out

Back to tasks

New task

Create a task with priority, status and optional project links.

Title

Testowe Zadanie

Description

Zadanie testowe

Priority

Low

Status

Do zrobienia

Project

No Project

Due date

15.07.2026

Tags

Testy Aplikacji

+ Add tag

Checklists

+ Add checklist

Lista Zakupów Biedronka

Sprzątanie

Test 2

Linked notes

Praca Inżynierska

Test

Create task

Rys. 18. Widok, przedstawiający formularz kreacji taska.

Rysunki 17–18 odnoszą się do scenariusza utworzenia taska. Lista tasków wykorzystuje stały układ aplikacji z bocznym menu nawigacyjnym i nagłówkiem sekcji, dzięki czemu użytkownik od razu widzi, w którym module pracuje. Nad kartami z taskami umieszczono pasek filtrów, sortowania i wyszukiwania, a przycisk utworzenia nowego taska wyróżniono kolorem akcentu w prawym górnym rogu widoku. Same taski zostały przedstawione w formie kart zawierających nazwę, status, priorytet, tagi oraz krótki opis. Formularz tworzenia taska zachowuje tę samą hierarchię wizualną: najpierw eksponowane są pola tytułu i opisu, następnie parametry organizacyjne, takie jak priorytet, status, projekt i termin realizacji. Niżej znajdują się tagi, powiązane checklisty oraz notatki, co porządkuje dane od najważniejszych informacji opisowych do relacji z innymi elementami systemu.

Piotr Słupski
Task Manager

Dashboard
Notes
Checklists
Tasks
Projects
Settings

Sign out

← Back to projects

New project

Create a project with priority, lifecycle status and linked checklists.

Title

Description

Priority: Medium Status: Active

Due date: dd-mm-yyyy

Tags: Tag 1 + Add tag

Project checklists: + Add checklist

- ☐ Lista Zakupów Biedronka
- ☐ Sprzątanie
- ☐ Test 2

Linked notes:

- ☐ Praca Inżynierska
- ☐ Test

Project tasks: + Add task

Rys. 19. Widok przedstawiający pierwszą część formularza kreacji projektu.

Projects

Settings

Sign out

Project tasks: + Add task

Add tasks here to create them together with this project.

Kanban columns: + Add column

Column title	Color	Done	Up	Down	Delete
Backlog	[Grey]	☐	↑	↓	🗑️
To do	[Blue]	☐	↑	↓	🗑️
In progress	[Orange]	☐	↑	↓	🗑️
Testing	[Purple]	☐	↑	↓	🗑️
Done	[Green]	☒	↑	↓	🗑️

Create project

Rys. 20. Widok przedstawiający drugą część formularza kreacji projektu.

Project tasks

+ Add task

Task title	Priority	Status	Due date		
Zadanie 1	Medium	In Progress	dd.mm.rrrr		
Zadanie 2	Medium	Backlog	dd.mm.rrrr		

Rys. 21. Widok, przedstawiający opcję dodawania tasków do projektu.

P

Back to projects

Projekt Testowy Aplikacji

Status: Paused | Priority: High

2 tasks

Created: 18.07.2026, 09:53:42 | Updated: 18.07.2026, 09:53:42

Projekt Testowy

test

Kanban

List

Expand kanban

Kanban board

2 tasks across 5 columns

Backlog

1 task

Zadanie 2

medium

Back

Next

To do

0 tasks

No tasks in this column.

In progress

1 task

Zadanie 1

medium

Back

Next

Testing

0 tasks

No tasks in this column.

Done

0 tasks

No tasks in this column.

New task

Rys. 22. Widok, przedstawiający tablicę kanban projektu.

Piotr Skupski

Task Manager

Dashboard

Notes

Checklists

Tasks

Projects

Settings

Back to projects

Projekt Testowy Aplikacji

Status: Paused | Priority: High

2 tasks

Created: 18.07.2026, 09:53:42 | Updated: 18.07.2026, 09:55:27

Projekt Testowy

test

Kanban

List

Task list

2 tasks

Zadanie 2

medium

Backlog

Zadanie 1

medium

In progress

New task

Rys. 23. Widok, przedstawiający taski projektów w formie listy.

Rysunki 19–23 prezentują interfejs związany ze scenariuszem utworzenia projektu i zarządzania podzadaniami. Formularz projektu został podzielony na logiczne części, co ogranicza wizualne przeciążenie mimo dużej liczby parametrów. W pierwszej części użytkownik

uzupełnia podstawowe dane, takie jak tytuł, opis, priorytet, status, termin, tagi oraz powiązane checklisty i notatki. Druga część formularza obejmuje konfigurację kolumn kanban, w której każda kolumna jest pokazana jako osobny wiersz z nazwą, kolorem, oznaczeniem zakończenia oraz przyciskami zmiany kolejności i usunięcia. Sekcja dodawania tasków do projektu wykorzystuje układ tabelaryczny, dzięki czemu kilka podzadań można porównać w jednym poziomym rzędzie według tytułu, priorytetu, statusu i terminu.

Po utworzeniu projektu użytkownik może wybrać jeden z dwóch sposobów przeglądania tasków. Widok kanban eksponuje kolumny statusów jako równoległe obszary robocze, z kolorowymi oznaczeniami nagłówków i paskami postępu. Karty tasków znajdują się bezpośrednio w kolumnach odpowiadających ich statusom, a przyciski przejścia do poprzedniej lub następnej kolumny wzmacniają czytelność procesu. Alternatywny widok listy przedstawia te same taski w bardziej zwartym układzie, przeznaczonym do szybkiego przeglądania nazw, priorytetów oraz aktualnych statusów. Przełącznik pomiędzy widokiem kanban i listą jest widoczny nad obszarem tasków, dzięki czemu użytkownik może dobrać prezentację danych do aktualnego sposobu pracy.

7. Implementacja kodu frontendowego

W niniejszym rozdziale opisano wybrane elementy implementacji warstwy frontendowej aplikacji. Skupiono się na dwóch częściach systemu, które mają szczególne znaczenie dla ergonomii pracy użytkownika: widoku kanban w projekcie oraz sekcji przypiętych elementów na dashboardzie. Oba fragmenty zostały zaimplementowane jako komponenty React działające w środowisku Next.js i komunikujące się z backendem za pomocą endpointów API.

7.1 Implementacja widoku kanban

Widok kanban został zaimplementowany głównie w komponencie `ProjectKanbanBoard`, znajdującym się w pliku `project-kanban-board.tsx`. Komponent ten odpowiada za wyświetlanie tablicy projektu, kolumn statusów oraz kart zadań. Dane wejściowe przekazywane do komponentu obejmują identyfikator projektu, listę kolumn oraz listę zadań. Dzięki temu komponent nie pobiera danych samodzielnie, lecz otrzymuje je z warstwy strony projektu, co upraszcza jego odpowiedzialność i pozwala wykorzystać renderowanie po stronie serwera dla początkowego stanu widoku.

Każda kolumna kanban jest reprezentowana przez obiekt zawierający identyfikator, nazwę, kolor oraz informację, czy dana kolumna oznacza zadanie zakończone. Zadanie zawiera między innymi identyfikator, tytuł, opis, priorytet, aktualny status, pozycję, termin oraz tagi. Relacja pomiędzy zadaniem i kolumną jest realizowana przez pole `statusId`. Jeżeli `statusId` zadania jest równe identyfikatorowi kolumny, karta zadania zostaje wyrenderowana właśnie w tej kolumnie.

Komponent utrzymuje lokalny stan tablicy za pomocą hooków `useState`. W stanie przechowywane są między innymi aktualne zadania, kolumny, identyfikator przenoszonego zadania, informacja o aktywnej kolumnie upuszczania oraz dane edytowanej kolumny. Dodatkowo zastosowano `useEffect`, aby synchronizować lokalny stan komponentu z danymi przekazanymi przez stronę projektu. Dzięki temu po odświeżeniu danych przez router widok zostaje zaktualizowany zgodnie ze stanem zapisanym w bazie.

Lista kolumn jest wyliczana w `useMemo`. Jeżeli projekt nie posiada zdefiniowanych kolumn, komponent tworzy kolumnę domyślną `todo`. Dodatkowo sprawdzane są statusy istniejących zadań. Jeżeli zadanie posiada status, którego nie ma w konfiguracji projektu, tworzona jest dodatkowa kolumna techniczna. Takie rozwiązanie zabezpiecza interfejs przed sytuacją, w której zadanie miałoby zniknąć z tablicy tylko dlatego, że jego status nie występuje już w

konfiguracji kolumn.

Przenoszenie zadania między kolumnami zostało zaimplementowane na dwa sposoby. Pierwszym jest mechanizm drag and drop, w którym funkcje `handleDragStart`, `handleDragOver` i `handleDrop` obsługują przeciąganie karty oraz upuszczanie jej w wybranej kolumnie. Drugim sposobem są przyciski `Back` i `Next`, które przenoszą zadanie do poprzedniej lub następnej kolumny. Dzięki temu widok pozostaje użyteczny również wtedy, gdy użytkownik nie chce korzystać z przeciągania.

Za właściwą zmianę statusu odpowiada funkcja `moveTask`. Po wybraniu nowej kolumny funkcja oblicza następną pozycję zadania w tej kolumnie, aktualizuje lokalny stan tablicy i wysyła żądanie `PATCH` do endpointu `/api/tasks/{taskId}`. W treści żądania przekazywane są `statusId`, `projectId` oraz `position`. Aktualizacja lokalnego stanu wykonywana jest przed odpowiedzią serwera, co daje użytkownikowi wrażenie natychmiastowej reakcji interfejsu. Jeżeli backend zwróci błąd, komponent przywraca poprzedni stan zadań i wyświetla komunikat o niepowodzeniu operacji.

Istotnym elementem implementacji jest również edycja kolumn. Użytkownik może kliknąć nagłówek kolumny, zmienić jej nazwę, kolor oraz oznaczyć ją jako kolumnę końcową. Po zapisaniu zmian komponent wysyła żądanie `PATCH` do endpointu `/api/projects/{projectId}`, przekazując zaktualizowaną tablicę `kanbanColumns`. Każda kolumna otrzymuje także pozycję wynikającą z kolejności w tablicy. Po poprawnym zapisie wykonywane jest `router.refresh()`, co odświeża dane projektu.

Widok kanban jest osadzony w komponencie `ProjectTaskView`. Ten komponent pozwala przełączać sposób prezentacji zadań między widokiem kanban i widokiem list. Wybrany tryb jest zapisywany w projekcie przez żądanie `PATCH` do endpointu projektu, w polu `taskView`. Dzięki temu wybór użytkownika nie jest wyłącznie stanem tymczasowym w przeglądarce, ale częścią konfiguracji projektu przechowywaną w bazie danych. Dodatkowo dla tablicy kanban dostępny jest tryb rozszerzony, który wyświetla tablicę na całym ekranie i ułatwia pracę z większą liczbą zadań.

Karty zadań w widoku kanban zawierają najważniejsze informacje potrzebne do szybkiej oceny pracy: tytuł, skrócony opis, priorytet, termin oraz tagi. Priorytety są wyróżnione kolorami, a termin jest prezentowany razem z ikoną kalendarza. Każda karta jest linkiem do szczegółów zadania, dzięki czemu użytkownik może szybko przejść z ogólnego widoku projektu do edycji konkretnego elementu.

7.2 Implementacja sekcji przypiętych elementów dashboardu

Sekcja przypiętych elementów dashboardu została oparta na kolekcji `pins`, która przechowuje identyfikator użytkownika, typ przypiętego elementu, identyfikator celu oraz pozycję. Frontend nie przechowuje osobnych list przypiętych notatek, zadań, checklist i projektów. Zamiast tego każdy pin zawiera parę `targetType` oraz `targetId`, która pozwala wskazać dowolny

obsługiwany typ elementu.

Pobieranie danych do dashboardu odbywa się w pliku `dashboard/page.tsx`. Strona najpierw sprawdza sesję użytkownika, a następnie łączy się z bazą danych. Za pomocą `Promise.all` równolegle pobierane są przypięcia, liczniki głównych typów danych, ostatnio aktualizowane notatki, checklisty, zadania i projekty oraz wydarzenia kalendarza wynikające z terminów zadań i projektów. Takie podejście ogranicza czas oczekiwania na dane, ponieważ niezależne zapytania są wykonywane równocześnie.

Dla każdego dokumentu Pin wykonywana jest funkcja `findPinnedTarget`. Funkcja ta sprawdza typ przypięcia i na tej podstawie pobiera właściwy dokument z kolekcji `notes`, `checklists`, `tasks` albo `projects`. Każde zapytanie zawiera warunek `ownerId` oraz `archivedAt`: `null`, dlatego dashboard pokazuje tylko aktywne elementy należące do zalogowanego użytkownika. Jeżeli przypięty dokument nie istnieje lub został zarchiwizowany, nie jest przekazywany do interfejsu.

Po pobraniu celu przypięcia dane są mapowane do wspólnego formatu `PinnedItem`. Obiekt ten zawiera tytuł, opis, typ, status, priorytet, tagi, daty utworzenia i aktualizacji oraz link prowadzący do szczegółów elementu. Dzięki ujednoliceniu danych komponenty frontendowe mogą renderować przypięte notatki, checklisty, zadania i projekty w jednej sekcji, bez konieczności tworzenia czterech osobnych widoków.

Głównym komponentem dashboardu jest `PinnedBoard`. Renderuje on kafelki metryk, mały kalendarz oraz komponent `PinnedItemsSearch`. Kafelki metryk pokazują liczbę notatek, checklist, zadań i projektów. Po najechnięciu na kafelek wyświetlana jest lista ostatnio aktualizowanych elementów danego typu. Kalendarz jest budowany lokalnie na podstawie zadań i projektów mających termin w bieżącym miesiącu. Funkcja `buildCalendarDays` tworzy siatkę dni, oznacza aktualny dzień oraz przypisuje wydarzenia do odpowiednich dat.

Za właściwą listę przypiętych elementów odpowiada komponent `PinnedItemsSearch`. Komponent ten działa po stronie klienta, ponieważ obsługuje stan wyszukiwarki, filtrów oraz sortowania. Użytkownik może wyszukiwać przypięte elementy po tytule, opisie, typie, statusie i metadanych. Może także filtrować wyniki według typu elementu: notatek, checklist, zadań lub projektów. Lista typów jest przechowywana w stanie `selectedTypes`, a filtrowanie wykonywane jest w `useMemo`, aby nie przeliczać wyników bez potrzeby przy każdym renderowaniu.

Sortowanie przypiętych elementów obsługuje stan `sortField` oraz `sortDirection`. Użytkownik może sortować elementy po dacie aktualizacji, dacie utworzenia, tytule lub opisie. Dla pól tekstowych domyślnym kierunkiem jest sortowanie rosnące, natomiast dla dat domyślnie pokazywane są najnowsze elementy. Wyniki po filtrowaniu są kopiowane do nowej tablicy i sortowane bez modyfikowania danych wejściowych przekazanych do komponentu.

Każdy przypięty element jest renderowany jako karta `ObjectCard`. Ikona karty zależy od typu elementu: notatka, checklist, zadanie lub projekt. Dla zadań i projektów przekazywany jest status oraz priorytet, natomiast dla checklist możliwe jest pokazanie podglądu elementów checklisty. Link karty prowadzi bezpośrednio do szczegółowego widoku danego elementu.

Za przypinanie i odpinanie odpowiada komponent przycisku pinezki, wydzielony w kodzie jako `PinEntityButton`. Komponent otrzymuje typ i identyfikator elementu oraz opcjonalny identyfikator istniejącego przypięcia. Jeżeli element nie jest przypięty, kliknięcie wysyła żądanie POST do endpointu `/api/pins`. Jeżeli element jest już przypięty, kliknięcie wysyła żądanie DELETE do endpointu `/api/pins/{pinId}`. Po poprawnym zakończeniu operacji komponent aktualizuje lokalny stan, zmienia wygląd ikony pinezki oraz wykonuje `router.refresh()`, aby odświeżyć dashboard i pozostałe widoki.

W efekcie sekcja przypiętych elementów pełni rolę spersonalizowanego skrótu do najważniejszych danych użytkownika. Implementacja została oparta na jednym wspólnym modelu przypięcia oraz na uniwersalnym komponencie listującym, dzięki czemu dodanie obsługi kolejnego typu elementu wymagałoby głównie rozszerzenia mapowania typów i sposobu pobierania celu przypięcia.

Bibliografia

- [1] Atlassian, *Trello*, <https://trello.com> (dostęp: 17.07.2026).
- [2] Asana, Inc., *Asana*, <https://asana.com> (dostęp: 17.07.2026).
- [3] ClickUp, *ClickUp*, <https://clickup.com> (dostęp: 17.07.2026).
- [4] Dean J., *Web programming with HTML, CSS, and JavaScript*, Burlington 2017.
- [5] React, *React: The library for web and native interfaces*, <https://react.dev> (dostęp: 28.01.2026).
- [6] Tailwind Labs, *Tailwind CSS*, <https://tailwindcss.com> (dostęp: 28.01.2026).
- [7] Vercel, *Next.js Docs*, <https://nextjs.org/docs> (dostęp: 29.01.2026).
- [8] Auth.js, *Auth.js*, <https://authjs.dev> (dostęp: 17.07.2026).
- [9] *Frontend vs Backend*, <https://d1zruf9db62p8s.cloudfront.net/2025/08/Frontend-vs-Backend.webp> (dostęp: 02.02.2026).
- [10] MongoDB, Inc., *Welcome to the MongoDB docs*, <https://www.mongodb.com/docs/> (dostęp: 02.02.2026).
- [11] Albiorix Technology, *Web application architecture layers*, <https://www.albiorixtech.com/wp-content/uploads/2025/12/web-application-architecture-layers.jpg> (dostęp: 03.02.2026).
- [12] Canva, *Creating a data flow diagram: How-tos, templates, and tips*, <https://www.canva.com/online-whiteboard/data-flow-diagrams/> (dostęp: 03.02.2026).

Spis rysunków

1	Tabela, porównująca Trello, Asanę oraz ClickUp.	6
2	fragment kodu przedstawiający definiowanie komponentu Button w czystym JavaScript, bez nakładki JSX	12
3	fragment kodu przedstawiający deklaratywne definiowanie komponentu Button dzięki JSX	13
4	fragment kodu przedstawiający klasowy komponent “Button”	13
5	fragment kodu przedstawiający funkcyjny komponent “Button”	14
6	przykład funkcji zarządzającej stanem komponentu “Counter”	15
7	przykładowe użycie TailwindCSS, pochodzące z oficjalnej strony frameworka (dostęp: 28.01.2026)	16
8	Przedstawienie procesu wyświetlania danych użytkownikowi aplikacji internetowej	24
9	Diagram warstw aplikacji internetowej	28
10	Prosty diagram komponentów internetowej aplikacji do zarządzania zadaniami.	29
11	Diagram przepływu danych poziomu 0, przedstawiający system jako jeden proces wraz z relacjami z otoczeniem.	30
12	Diagram przepływu danych poziomu 1	31
13	Diagram UML, wizualizujący scenariusz przypięcia taska	37
14	Widok, przedstawiający checklistę, z możliwością przypięcia jej do ekranu głównego.	41
15	Widok przedstawiający zachowanie przycisku “Przypnij” po przypięciu checklisty do ekranu głównego.	42
16	Widok przedstawiający elementy przypięte do ekranu głównego.	43
17	Widok, przedstawiający stronę z taskami użytkownika.	44
18	Widok, przedstawiający formularz kreacji taska.	45
19	Widok przedstawiający pierwszą część formularza kreacji projektu.	46
20	Widok przedstawiający drugą część formularza kreacji projektu.	46
21	Widok, przedstawiający opcję dodawania tasków do projektu.	47
22	Widok, przedstawiający tablicę kanban projektu.	47
23	Widok, przedstawiający taski projektów w formie listy.	47

Spis tabel

1	Przykładowa tablica kanban	8
---	--------------------------------------	---